DiariCore

Comprehensive System Documentation

A Progressive Web Application for private journaling
with transformer-based emotion classification

Technical record of the current implemented system

August 2026

Author

Tolentino, Lawrence Dave P.

Live application: https://diaricore.up.railway.app/

GitHub (workspace origin): https://github.com/lproject012125/diari-core

GitHub (README remote): https://github.com/0323-3621-cell/diaricore

Table of Contents

1. Project Overview

DiariCore is a private, account-based journaling Progressive Web App (PWA). Users write dated journal entries with optional titles, tags, and photos, then receive emotion and sentiment labels produced by a fine-tuned XLM-RoBERTa-Base transformer model. The labels are stored with each entry and drive dashboard summaries, insights charts, and journaling streaks. A Suggestions page is included for supportive reading; its content is currently static. The product is a self-reflection tool. It is not a medical or diagnostic system.

The problem the system addresses is that generic notes apps and social platforms do not combine private journaling, structured tags, mood pattern visualization, installable mobile-like behavior, and security controls suitable for personal writing. DiariCore focuses that workflow in one application that can be used in a browser or installed to the home screen.

Item	Current value
Project name	DiariCore
Project type	Full-stack web application and Progressive Web App
Intended users	Individuals who want a private journal with light mood tracking (students and general users)
Primary live host	Railway (Flask + Gunicorn + PostgreSQL)
Emotion inference host	Hugging Face Space (FastAPI + ONNX Runtime)
Current status	Deployed and operational on free-tier services, with known limitations documented later

The application is implemented as static HTML/CSS/JavaScript served by Flask, not as a separate SPA framework. Authenticated JSON APIs persist data in PostgreSQL on Railway (SQLite is used when DATABASE_URL is unset). Machine-learning inference is intentionally not loaded inside the Railway web process.

2. Objectives

2.1 General objective

To provide a secure, mobile-capable journaling system that stores private entries, classifies emotional tone using a transformer model hosted separately from the web app, and presents those results in dashboards, insights, and suggestions without treating the output as clinical diagnosis.

2.2 Specific objectives

- Support registration with privacy consent, email OTP verification, session login, optional Google Authenticator TOTP, password reset, and profile management.
- Provide journal CRUD with titles, body text, custom tags, optional images, and editable entry date/time.
- Classify entry text into five emotion classes (angry, anxious, happy, neutral, sad) plus a derived sentiment label.

- Surface patterns on Dashboard, Insights, Suggestions, and a streak indicator.
- Deliver PWA installability, service-worker caching, offline-tolerant drafts, and Web Push reminders.
- Protect accounts with password policy, CSRF checks, rate limits, login lockouts, and security headers.
- Keep the Railway web service lightweight by running XLM-RoBERTa inference on Hugging Face.

3. System Features and Functionalities

Features below exist in the current codebase. Later chapters explain behavior in detail.

3.1 Unauthenticated users

- Register with username, email, names, gender, birthday, and a password that must pass the shared strength policy.
- Agree to the privacy notice before the account request is accepted.
- Verify the account with a six-digit email OTP (Brevo) on the verification page.
- Sign in with username or email and password; complete TOTP if enabled.
- Recover a forgotten password with an email OTP, then set a new password.
- Recover TOTP-locked sign-in with a separate Brevo recovery OTP that disables authenticator until it is set up again.

3.2 Authenticated journal users

- Dashboard: today's emotion, weekly average, insight line, mood chart, recent entries, search, streak book.
- Write Entry: tags, title, body (default 300-word cap, aligned with the training-set length limit and planned for later expansion), date/time, photos, voice shortcut, save with analysis, optional re-analysis.
- Voice Entry: microphone capture, live captions where the browser supports Web Speech, on-device Whisper fallback, hand-off into Write Entry.
- Entries: month navigation, filters, search, open/edit/delete.
- Entry view: full entry, mood UI, edit and delete.
- Insights: weekly trend, emotion breakdown, emotion-by-tag, triggers, weekly snapshot, consistency metrics.
- Suggestions: a page of currently static support copy and activity ideas (not clinical advice). Personalization is planned as a later improvement.
- Profile: avatar, personal fields, email change OTP, password change OTP, TOTP, theme palettes, dark mode, notification time, privacy copy, logout.
- PWA install, splash/launch overlay, offline draft queue, and push subscription when installed and permitted.

3.3 Administrator

A user whose email matches DIARI_ADMIN_EMAIL can open /admin. The current admin UI includes a user list with a read-only detail view, service health (email and AI test actions), audit logs, and settings. Account disable and delete controls are not present on the admin interface. Admin is a configured operator role, not a public self-serve role.

3.4 Cross-cutting platform behavior

- Session cookies (HTTP-only, SameSite=Lax, 14-day lifetime; Secure on Railway/production).
- CSRF validation on cookie-authenticated state-changing API calls.
- API responses are marked no-store so JSON is not reused as a stale cache.
- Uploads persist under UPLOADS_DIR (intended for a Railway volume).

4. Technology Stack

Versions below are the minimums declared in requirements files. Exact pinned versions on a given deploy may be newer.

Category	Technology / service
Frontend	HTML5, CSS3, vanilla JavaScript (ES6+), Bootstrap 5.3 CSS/icons, Chart.js, Lottie
Backend	Flask 3+, Gunicorn 21+ (Procfile: gunicorn app:app -c gunicorn.conf.py)
Programming language	Python 3.10+ (3.12 recommended locally; HF Space Dockerfile uses Python 3.11)
Database	PostgreSQL on Railway (psycopg2); SQLite locally when DATABASE_URL is unset
Machine learning	Fine-tuned XLM-RoBERTa-Base (transformer sequence classifier with a custom head)
Frameworks / libraries	httpx, huggingface_hub, pyotp, segno, pywebpush, py-vapid, cryptography, Werkzeug
Authentication	Flask sessions, Werkzeug password hashes, CSRF header/origin checks
Two-factor authentication	TOTP (RFC 6238) via pyotp; Google Authenticator–compatible otpauth QR codes
Category	**Technology / service**
Email / OTP	Brevo transactional SMTP HTTP API (api.brevo.com/v3/smtp/email)
PWA	Web App Manifest, service worker at site root, Cache Storage, beforeinstallprompt
Security	Password policy, login lockouts, OTP resend limits, CSP (optional disable), security headers
Model training	Google Colab notebook DiariCore_Model_Final_Cleaned.ipynb (PyTorch, CUDA when available)
ML inference	Hugging Face Space FastAPI + ONNX Runtime; Hub artifacts including model.onnx
Voice (client)	Web Speech API; @xenova/transformers Whisper Tiny/Small in the browser
Voice (server fallback)	Hugging Face Inference router, default openai/whisper-large-v3-turbo
Scheduled jobs	In-process push dispatcher every 60s; optional external cron-job.org
Version control	Git / GitHub
Hosting	Railway (web + Postgres); Hugging Face Hub and Spaces

Historical note: README also lists an AWS EC2 URL. This documentation treats Railway as the current primary web deployment because that is how the live Procfile application is operated. EC2 is recorded as an additional or earlier hosting path, not as the ML inference host.

5. System Design and Architecture

5.1 Runtime architecture

The user interacts with HTML pages and client JavaScript. Flask on Railway authenticates the session, reads and writes PostgreSQL, stores images on a volume, sends email through Brevo, and for mood analysis posts JSON to the Hugging Face Space POST /predict endpoint. The Space loads tokenizer files and an ONNX graph from the Hub repository, runs CPU inference, applies calibration and a limited keyword override, and returns emotion and sentiment fields. The web app stores those fields on the journal_entries row. Google Authenticator never talks to DiariCore's servers; it only displays TOTP codes from a secret stored on the user row. Push uses VAPID Web Push to installed PWA devices.

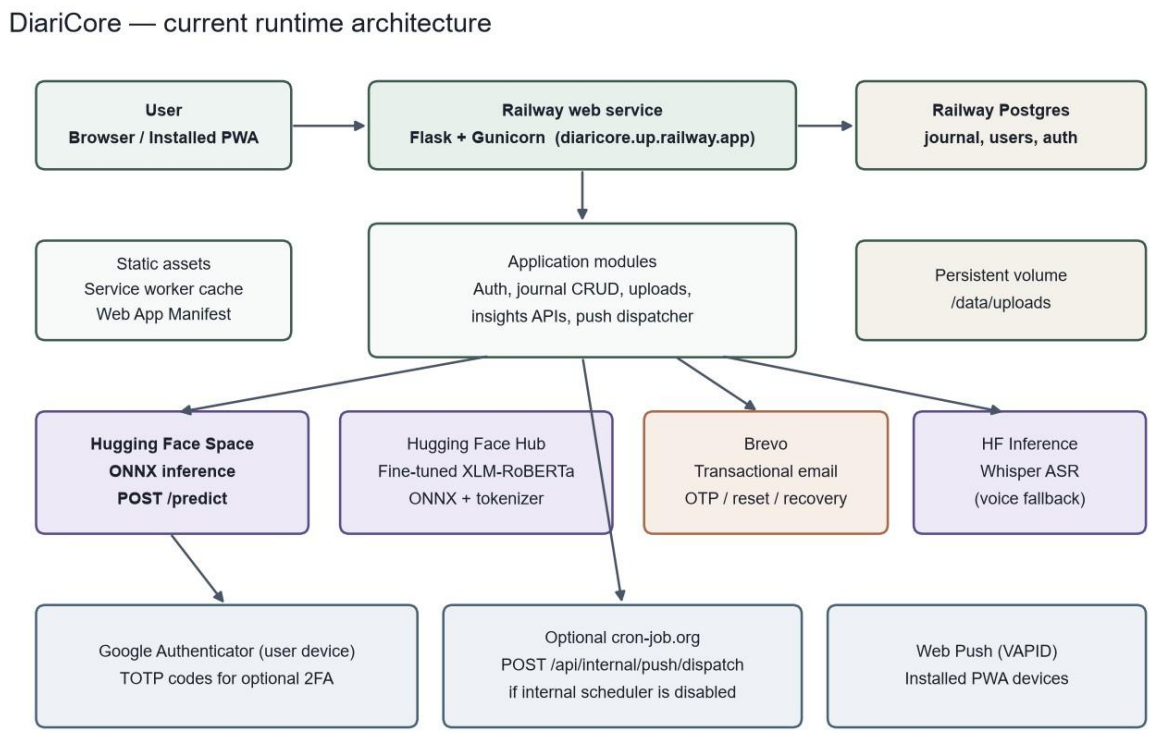


Figure 1. Current runtime architecture (Railway web app, Postgres, Hugging Face, Brevo, TOTP, optional cron).

5.2 Why Hugging Face holds the model

XLM-RoBERTa-Base ONNX weights are on the order of 1.1 GB (model.onnx), with a quantized variant around 279 MB. Loading that graph plus tokenizers in the same free-tier Railway process as Gunicorn, PostgreSQL clients, and file uploads is not practical: deploys become slow, memory pressure is high, and request latency suffers. The architectural decision is to keep Railway as the application and data plane and Hugging Face as the inference plane. Railway only needs outbound HTTP (space_nlp.analyze) with a 90-second timeout to tolerate Space cold starts.

5.3 Integrated service map

```
Dataset (text, label, language)          |          v
Google Colab  — fine-tune XLM-RoBERTa-Base          |
v Hugging Face Hub  — tokenizer + ONNX/PyTorch
artifacts          |          v Hugging Face Space  —
FastAPI /predict (ONNX Runtime)          |          v
Railway Flask app  — save / re-analyze journal text
        |
        +-- PostgreSQL  (users, entries, auth, push)
        +-- Brevo       (email OTP, reset, TOTP recovery)
        +-- Web Push    (VAPID to installed PWA)
        +-- Optional cron-job.org  (if internal dispatcher disabled)
```

6. Major Workflows

6.1 Registration and email OTP

POST /api/register validates fields, password policy, uniqueness, and privacy consent, hashes the password, and stores a pending_registrations row with a six-digit OTP that expires in 10 minutes. Brevo sends the code. The user opens verify-registration.html and submits POST /api/register/verify. A matching OTP creates the users row. Resend uses POST /api/register/resend, which is both IP rate-limited and subject to the per-account OTP resend lockout. If Brevo keys are missing locally, the server logs the code in “dev mode” and still returns success so local testing can continue.

6.2 Login and optional TOTP

POST /api/login checks IP rate limits and persistent login lockouts, then verifies username/email and password. Disabled accounts are rejected. If TOTP is not enabled, a session and CSRF token are established. If TOTP is enabled, the API returns a short-lived login_totp_challenges token instead of a session. The client then posts the six-digit authenticator code to POST /api/login/totp. pyotp verifies the code with a ±1 step window. Success creates the session and clears lockout counters. Failed passwords increment lockouts: five failures in 15 minutes lock the identifier for 15 minutes.

6.3 Journal save and emotion prediction

On Save, the client posts to POST /api/entries. The server normalizes text (strip, remove null bytes and angle brackets), enforces the word cap (ENTRY_WORD_MAX, default 300), stores tags and image URLs, then calls space_nlp.analyze(text). That function POSTs {"text": ...} to SPACE_URL/predict. A 200 response with emotionLabel is stored (emotion, sentiment, scores, all_probs_json). Non-200, timeout, or malformed JSON triggers a keyword fallback in space_nlp.py so the save still completes, with engine reported as fallback. Re-analysis of existing text uses POST /api/entries/analyze-text (rate-limited) or PATCH when the entry is updated.

The 300-word default cap is a dataset-driven constraint, not a product-design ceiling. Training texts in 1500_dataset_expanded.xlsx were prepared around this length (Colab word-count summaries show maxima near 300 words). The live journal field follows that limit so inference stays close to the fine-tuning distribution. Raising or removing the cap is recorded as a future improvement once the training set covers longer entries.

6.4 Voice input

Write Entry’s microphone control opens voice-entry.html. The page requests getUserMedia, shows a waveform, and starts Web Speech Recognition when the constructor exists (Chrome/Edge). Audio is also recorded. If live captions are weak, the client can run on-device Whisper (transformers.js). The transcript is stored in session storage and injected into the Write Entry textarea. Server-side POST /api/voice/transcribe exists as an authenticated fallback using the Hugging Face Inference API when a token is configured.

6.5 Image attachment

POST /api/uploads/image accepts images (JPEG, PNG, WebP, GIF, BMP, TIFF, AVIF, HEIC/HEIF). Authenticated users are limited to 25 uploads per 60 seconds. URLs of the form /uploads/<filename> are stored in journal_entries.image_urls_json. Removing images from an entry deletes unused files under UPLOADS_DIR. Maximum 10 images per entry (input_security.MAX_ENTRY_IMAGES).

6.6 Push reminder dispatch

After PWA install and notification permission, the service worker subscribes with the VAPID public key and POST /api/push/subscribe stores the endpoint. gunicorn post_fork starts push_scheduler, which every minute calls push_service.dispatch_due_notifications. Daily reminders fire in a window after the user’s reminder time (Asia/Manila) if no entry exists that calendar day. If DISABLE_INTERNAL_PUSH_CRON is set, an external scheduler must POST /api/internal/push/dispatch with X-Push-Cron-Secret.

7. Database Design

Production uses PostgreSQL via DATABASE_URL (Railway plugin). Local development uses SQLite at DATABASE_PATH (default diaricore.db; start scripts often use diaricore.local.db). db.init_db() creates core tables and then ALTER/CREATE helpers add columns introduced later (TOTP, avatars, UI preferences, all_probs_json, and so on).

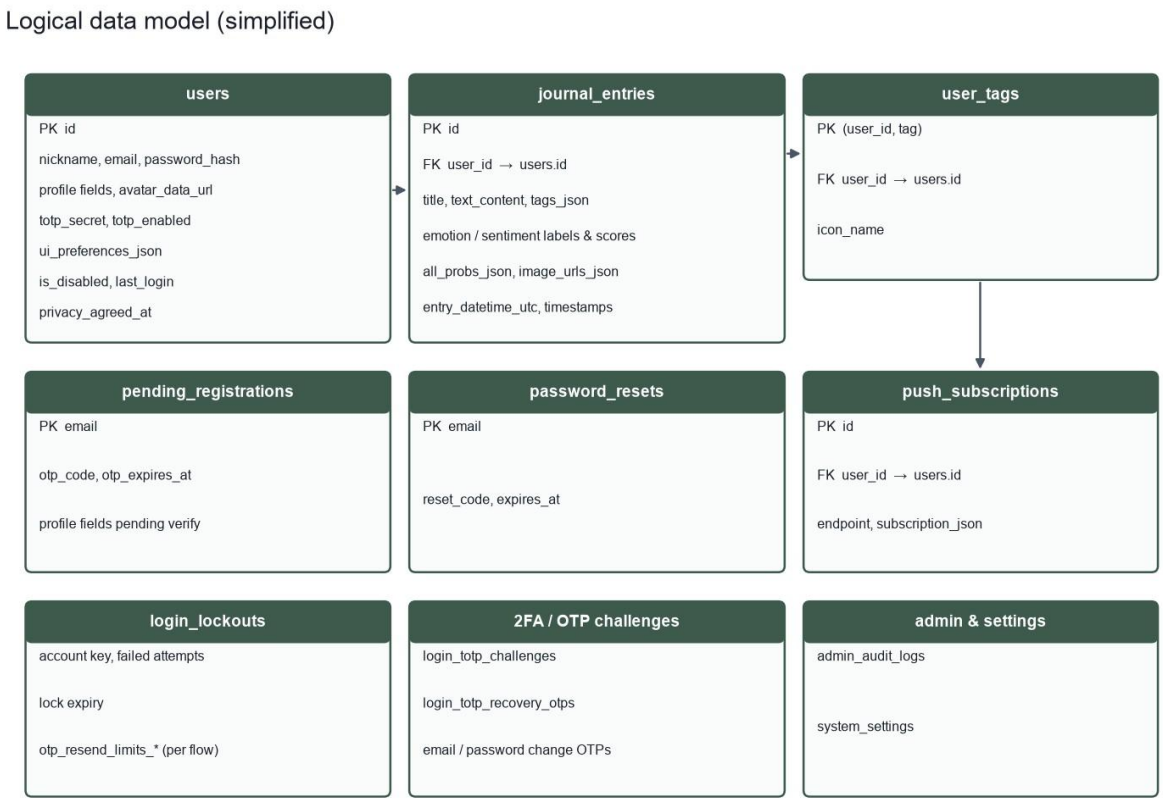


Figure 2. Logical data model (simplified). Secrets such as totp_secret are never sent to the browser except as required during TOTP setup.

Table	Role
users	Account identity, password_hash, profile, avatar_data_url, totp_*, ui_preferences_json, is_disabled, last_login, privacy_agreed_at
pending_registrations	Unverified sign-up rows keyed by email, including otp_code and otp_expires_at
password_resets	Forgot-password OTP keyed by email
journal_entries	Entry content, tags_json, mood fields, image_urls_json, timestamps; FK user_id
user_tags	Per-user tag vocabulary with optional icon_name; PK (user_id, tag)
login_totp_challenges	Short-lived tokens between password success and TOTP success
login_totp_recovery_otps	Email recovery codes that disable TOTP after verification
login_lockouts	Failed login timestamps and lock expiry per account key
otp_resend_limits_*	Per-flow OTP request windows (register, forgot, email change, password change, login recovery)
user_password_change_challenges	OTP challenge when changing password while logged in
user_profile_email_change_challenges	OTP challenge when changing email
push_subscriptions	Web Push endpoints and JSON for installed devices
system_settings	Optional overrides (Brevo, HF token, feature flags) used if env vars are empty
admin_audit_logs	Administrator actions

A Railway Data tab screenshot of project “daring-possibility” shows these tables present on production Postgres, including the split OTP resend-limit tables. The web service in that screenshot is associated with a persistent volume (web-volume) consistent with storing uploads outside the ephemeral container filesystem.

8. User Interface and Page Documentation

Authenticated pages share a desktop sidebar (Dashboard, Write Entry, Entries, Insights, Suggestions, Profile) and a mobile top bar with brand, search, and profile. Screenshots of live UI pages were not supplied with this documentation pass; placeholders mark where captures should be inserted. Backend screenshots that were supplied appear in later chapters.

8.1 Sign In (login.html)

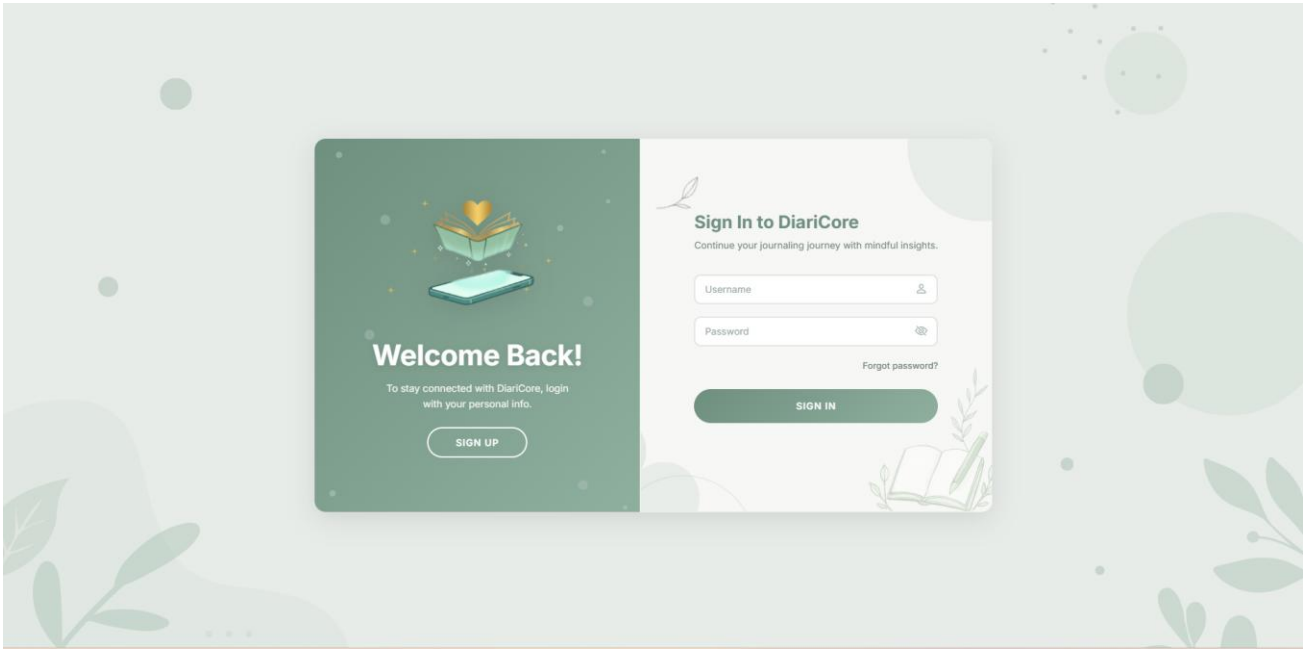
Purpose. Authenticate an existing user, start forgot-password, or continue to TOTP / recovery.

Access. Public. Installed PWA may skip this page if localStorage already has a logged-in user record.

Main UI. Branded split layout; username/email and password fields; Sign In; link to Register; in-page phases for forgot password, OTP, new password, and 2FA code.

Actions. Submit credentials; request reset code; enter TOTP; request authenticator recovery email.

Server interaction. POST /api/login, /api/login/totp, /api/login/totp/recovery/*, /api/password/forgot, /api/password/verify-code, /api/password/reset.



8.2 Register (register.html)

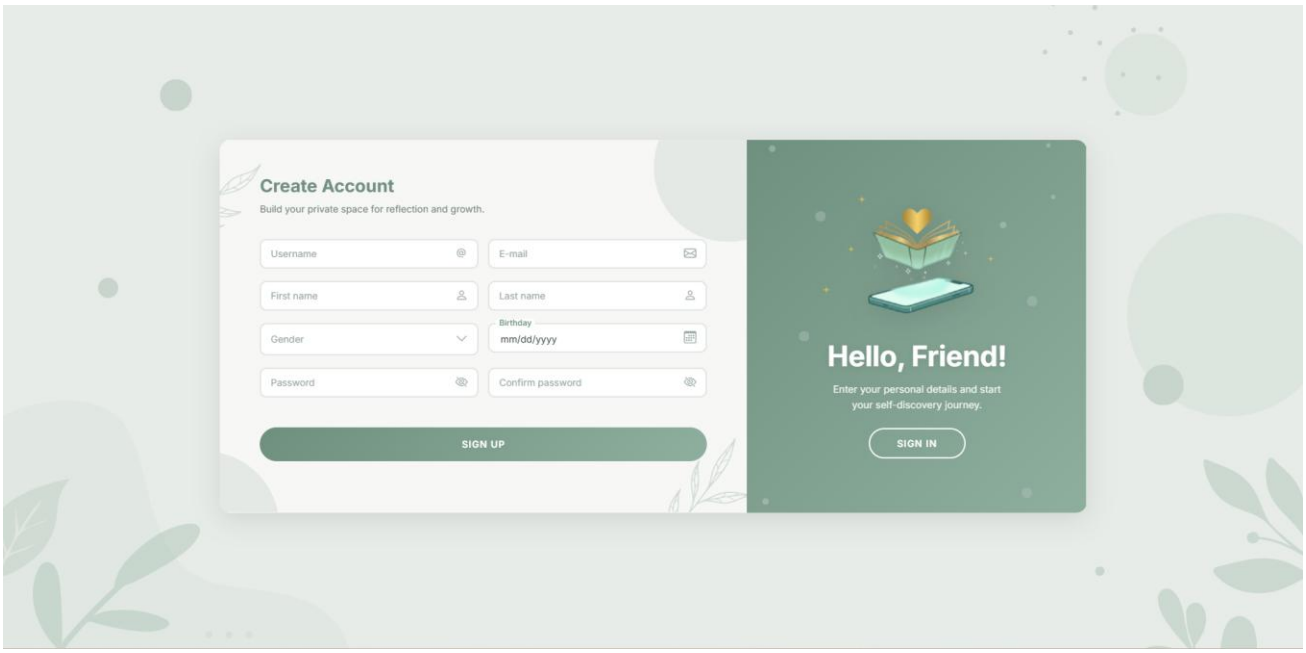
Purpose. Collect identity fields and consent, then create a pending registration.

Access. Public.

Main UI. Username, email, first/last name, gender, birthday, password with live rule checklist, privacy modal that must be accepted.

Actions. Validate client-side and POST /api/register; on success navigate to verification.

Server interaction. POST /api/register; GET /api/check-availability for username/email uniqueness.



8.3 Account Verification (verify-registration.html)

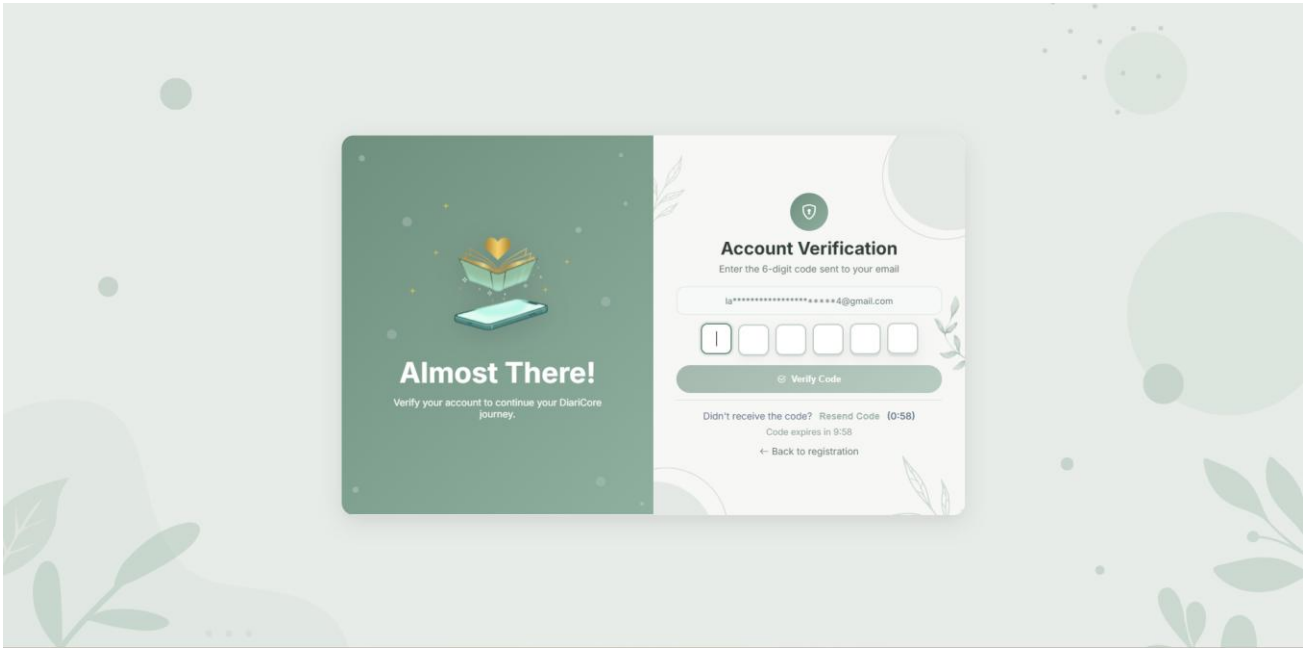
Purpose. Confirm the six-digit Brevo OTP and create the users row.

Access. Public users who just registered (email is known to the client).

Main UI. Six-digit inputs, resend control, branding panel.

Actions. Verify code; resend code; then sign in.

Server interaction. POST /api/register/verify and /api/register/resend. OTP expires in 10 minutes.



8.4 Dashboard (dashboard.html)

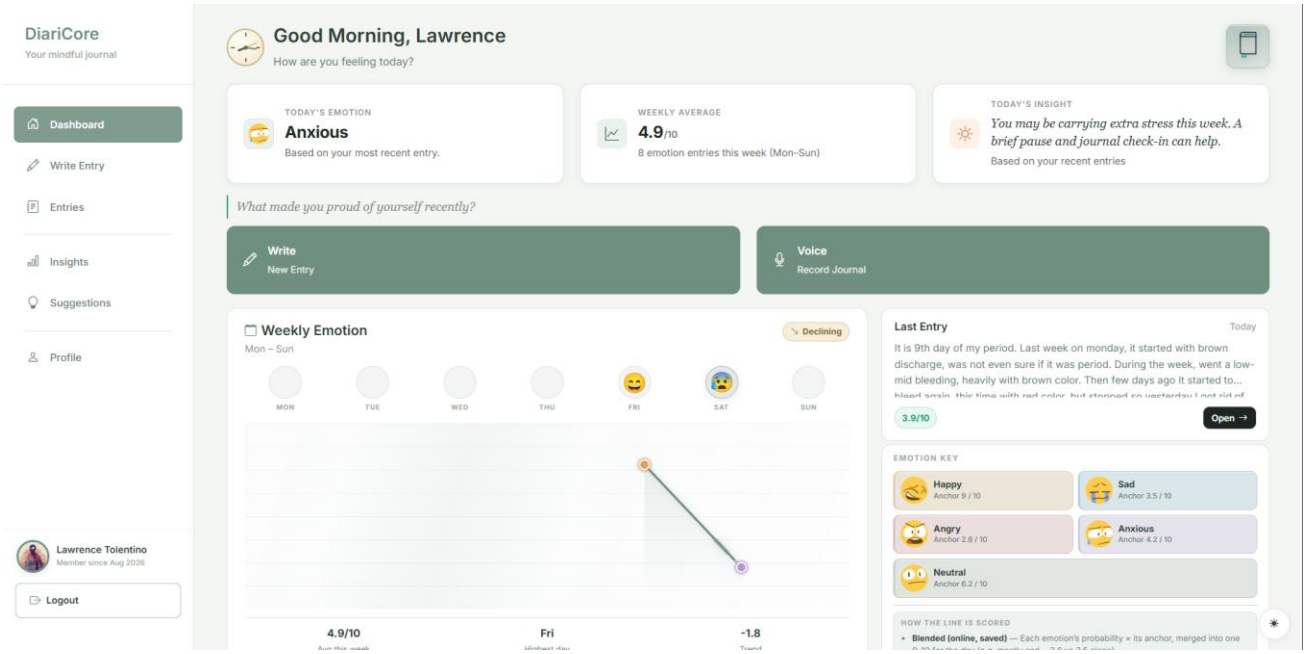
Purpose. Home summary of recent emotional tone, activity, and navigation into writing and history.

Access. Signed-in users (client redirects unauthenticated users to login).

Main UI. Stat cards (Today's Emotion, Weekly Average, Today's Insight), mood visualization, recent entry list, search/filter in the top bar, floating streak book (Lottie).

Actions. Open an entry, start a new entry, inspect streak, search entries.

Server interaction. GET /api/entries, /api/user/me, /api/sync/*, client-side mood-scoring and streak modules.



8.5 Write Entry (write-entry.html)

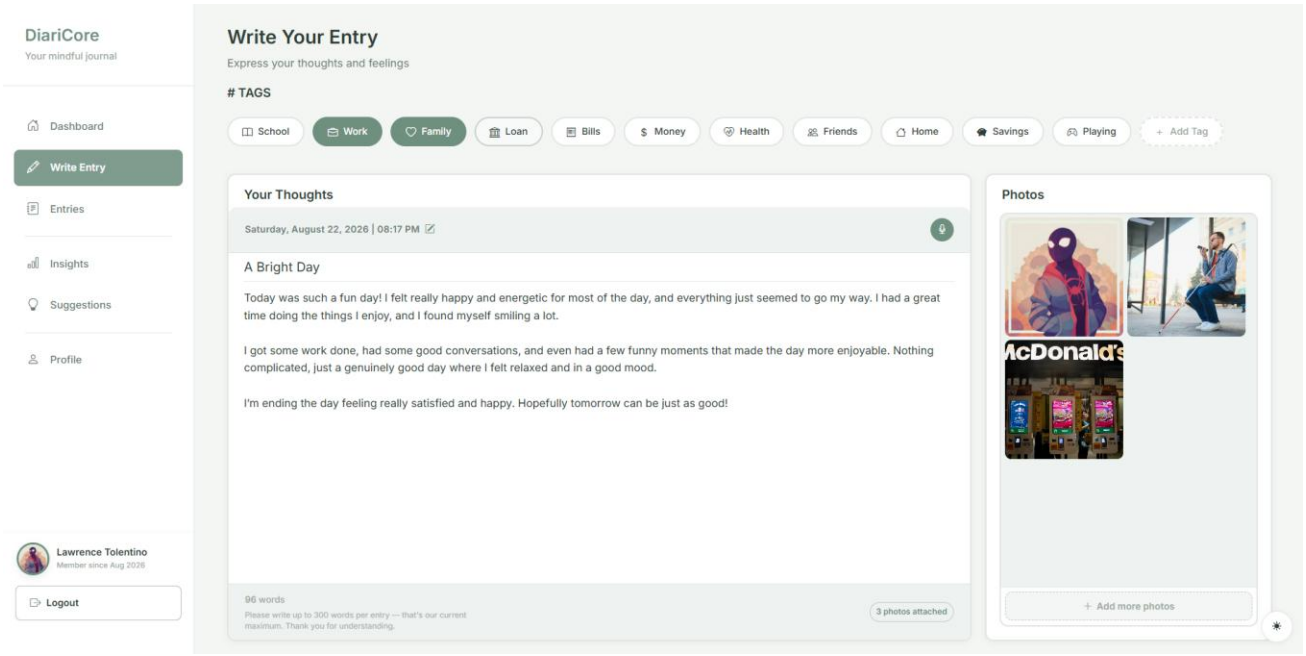
Purpose. Compose a new journal entry and request emotion analysis on save.

Access. Signed-in users.

Main UI. Tag chips (School, Home, Friends, Work, Family, Health, Money, plus user tags and Add Tag), title, textarea (default 300-word cap from the training-set length limit), date/time picker, mic button, photo picker, word counter, Save.

Actions. Select tags; attach up to 10 images; open Voice Entry; save; after save, view mood UI; re-analyze text when implemented on the client via /api/entries/analyze-text.

Server interaction. POST /api/entries, POST /api/uploads/image, POST /api/tags, POST /api/entries/analyze-text.



8.6 Voice Entry (voice-entry.html)

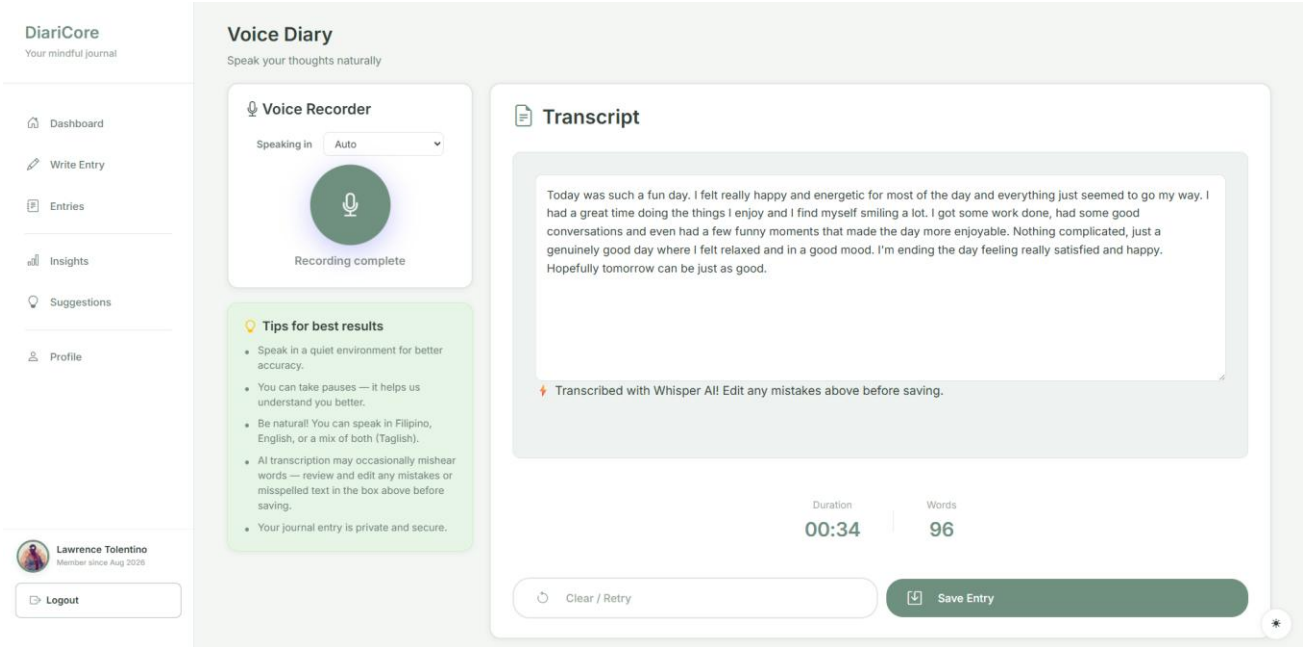
Purpose. Capture speech and produce text for the journal body.

Access. Signed-in users, typically from Write Entry.

Main UI. Large microphone control, recording timer, waveform, transcript box, language handling via voice-locale.js, first-run notice modal.

Actions. Start/stop recording; edit transcript; continue to Write Entry (session key diariCoreVoiceDraftForWrite).

Server interaction. Browser Web Speech and/or on-device Whisper; optional POST /api/voice/transcribe.



8.7 Entries (entries.html)

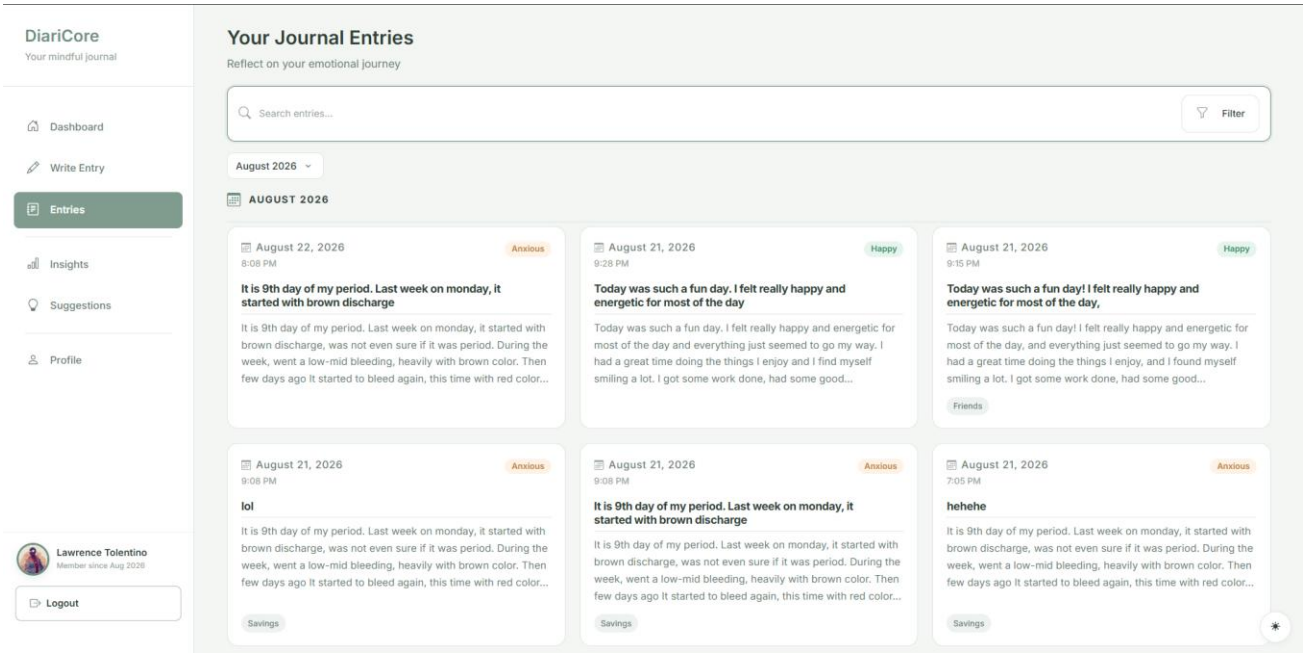
Purpose. Browse, filter, and open the journal archive.

Access. Signed-in users.

Main UI. Month navigation, mood/tag filters, search, entry cards; can restore a detail view via ?entryId=.

Actions. Open, edit, or delete; filter by month/mood/tag.

Server interaction. GET/PATCH/DELETE /api/entries and /api/entries/.



8.8 Entry View (entry-view.html)

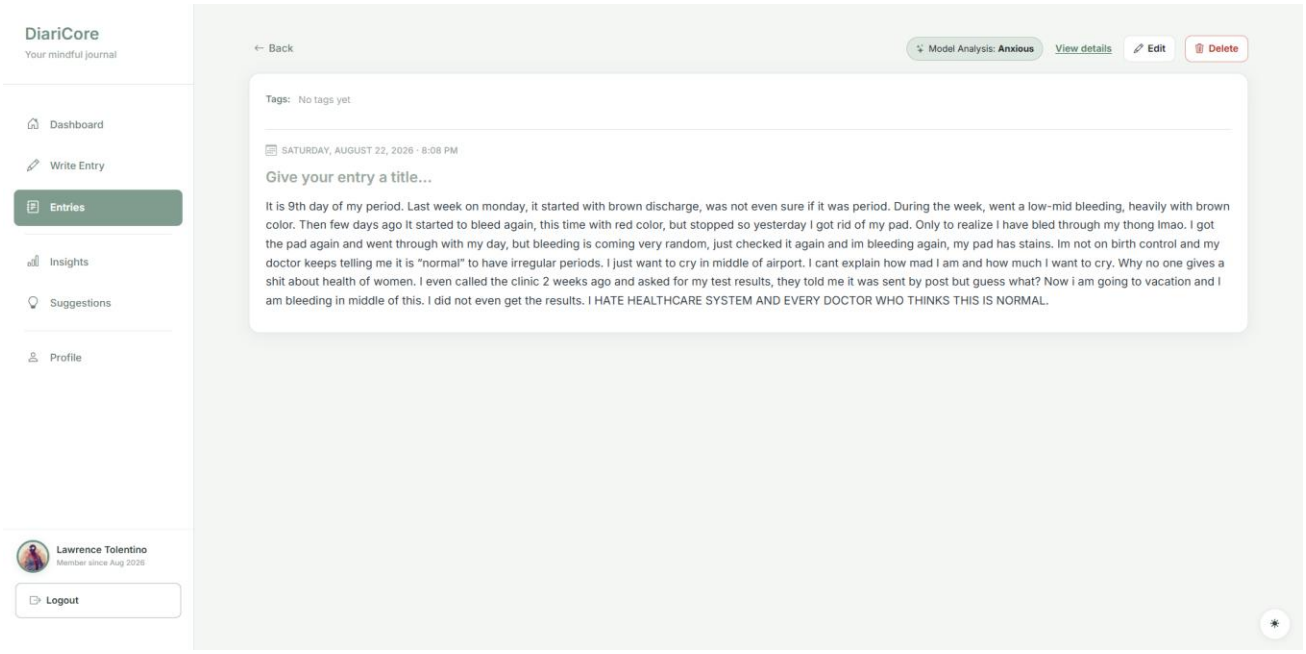
Purpose. Read one entry with stored mood results and edit or delete it.

Access. Signed-in users, for an entry they own.

Main UI. Title, datetime, tags, body, images, mood analysis panel.

Actions. Edit fields and save; delete; re-run analysis when the client posts analyze-text or PATCH.

Server interaction. GET/PATCH/DELETE /api/entries/.



8.9 Insights (insights.html)

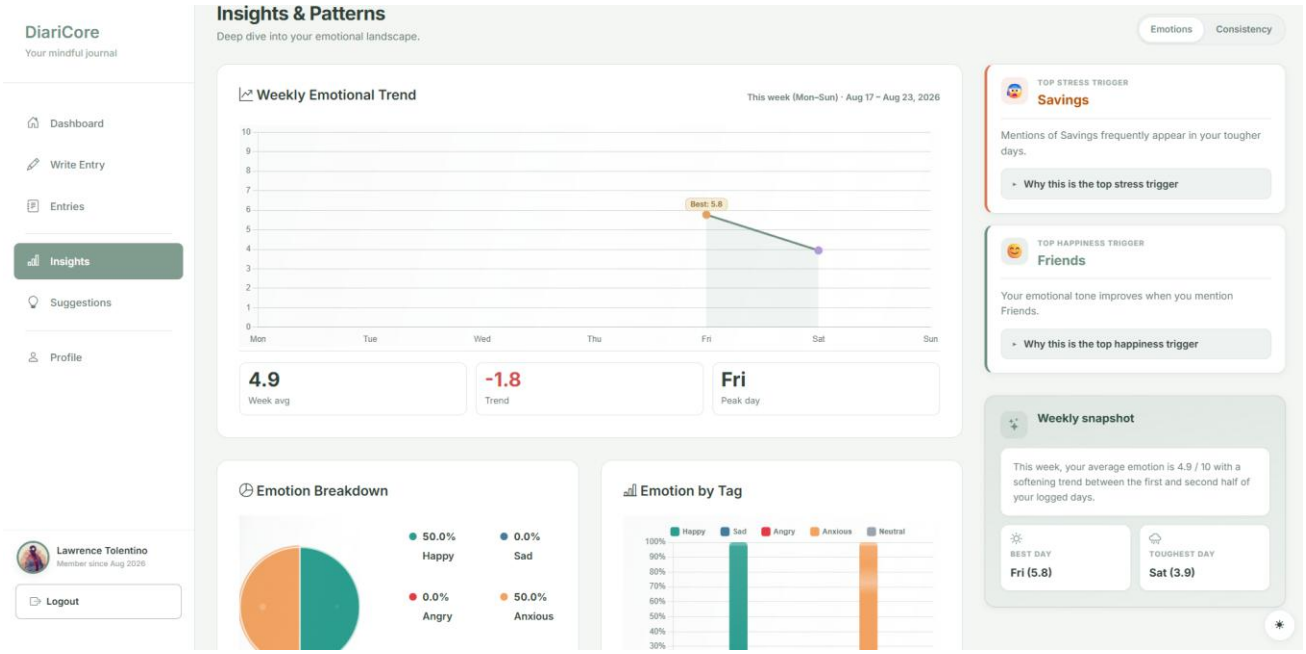
Purpose. Visualize emotion over time, by tag, and journaling consistency.

Access. Signed-in users.

Main UI. Weekly Emotional Trend charts, Emotion Breakdown pie, Emotion by Tag bars, trigger cards, weekly snapshot chips, consistency KPIs (active days, rate, entries/week, most active Asia/Manila hour), month selector.

Actions. Switch range/month; review triggers derived from tags on stressed vs happy entries.

Server interaction. GET /api/entries and GET /api/triggers/summary; Chart.js rendering in insights.js.



8.10 Suggestions (suggestions.html)

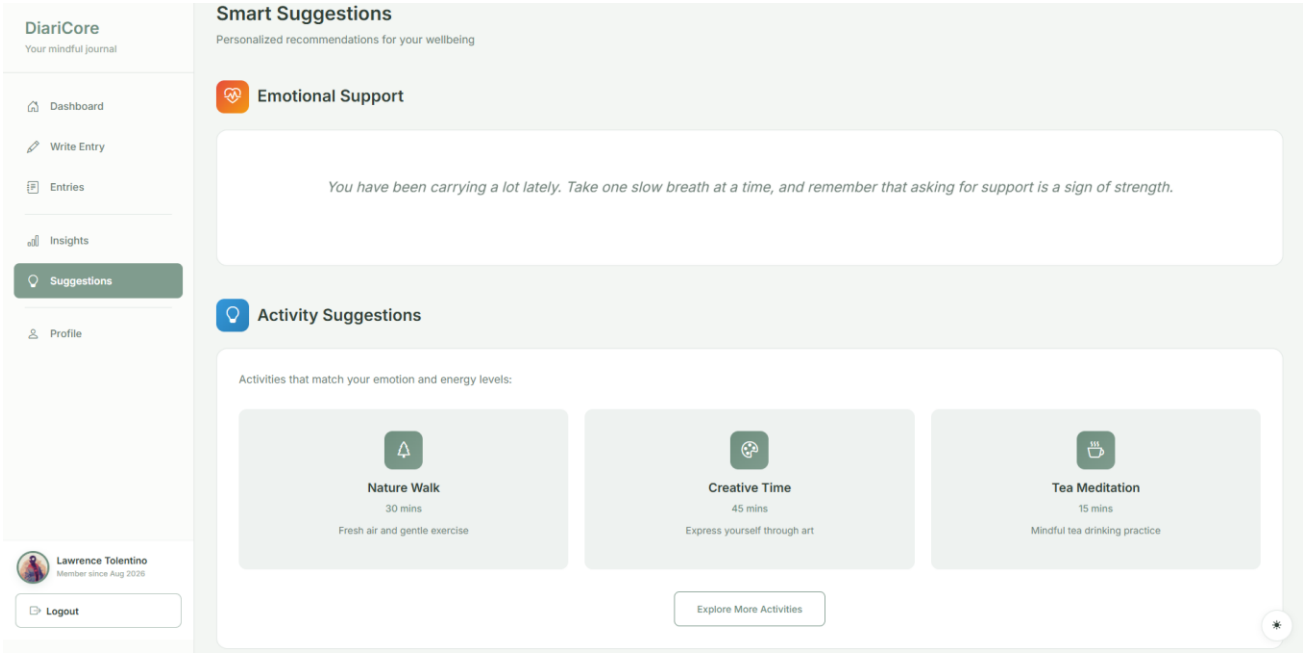
Purpose. Present supportive reading and activity ideas. This is coaching-style copy, not therapy. Page content is currently static and is planned for later improvement.

Access. Signed-in users.

Main UI. Emotional Support section and recommendation / activity cards defined in suggestions.html. A short support line may change among a few pre-written sentences; the cards, activities, and tips themselves are authored in the template rather than generated per user.

Actions. Read the static suggestions; navigate back to write or insights.

Server interaction. No dedicated recommendation API or model. suggestions.js attaches click handlers and a small localStorage-based support-line swap. Expanding this into personalized, dynamic suggestions is a future improvement.



8.11 Profile (profile.html)

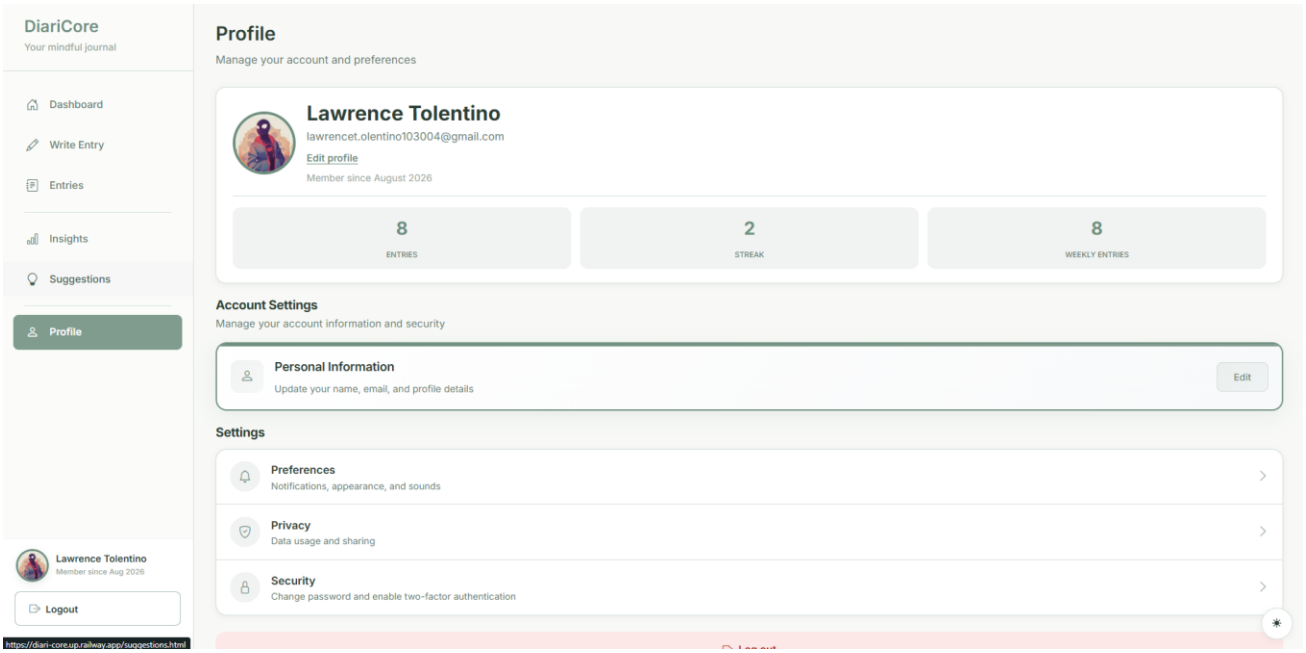
Purpose. Manage identity, security, appearance, and notification preferences.

Access. Signed-in users.

Main UI. Overview (avatar, name, member since), Account Settings, Settings hub. Nested panels: Appearance (dark mode, palettes theme-1…theme-10), Notifications (daily reminders, reminder time), Privacy, Personal info, email change, password change, Authenticator app 2FA modal.

Actions. Update profile; change email/password with OTP; enable/disable TOTP; set reminder time; logout.

Server interaction. GET/POST /api/user/me, /api/user/profile, avatar, ui-preferences, email-change-*, password change-*, totp/*, push/preferences.



8.12 Admin (admin.html at /admin)

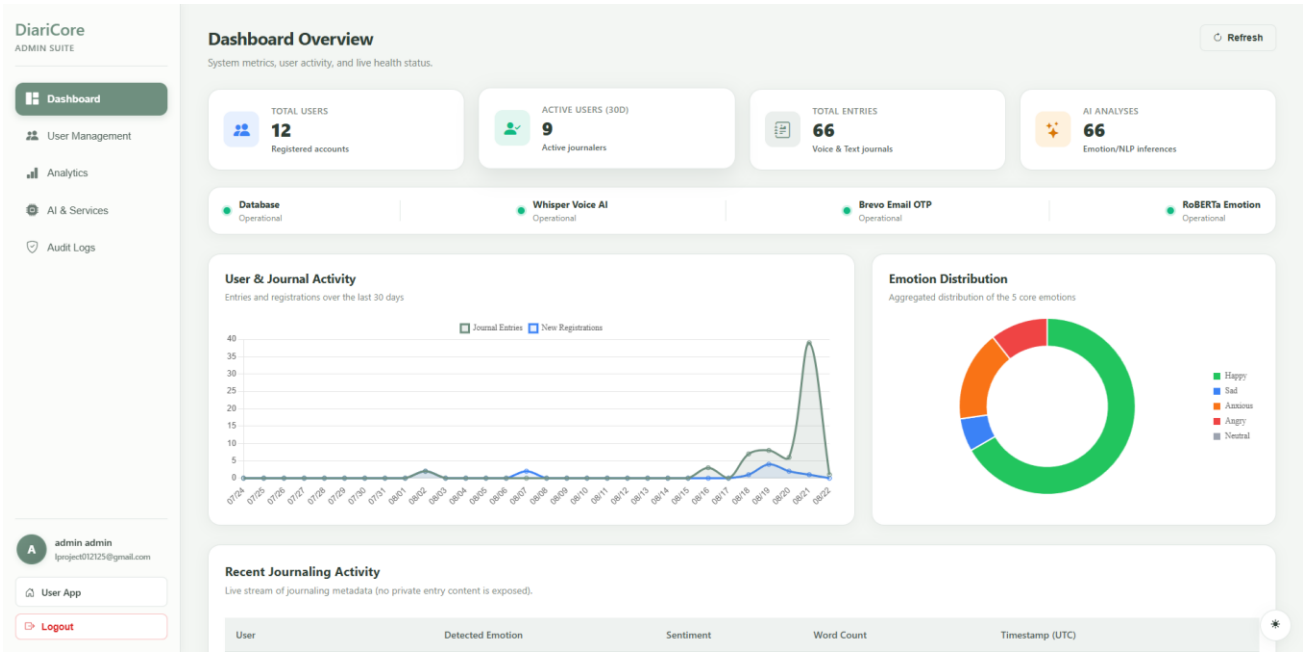
Purpose. Operate the deployment: users, service tests, audit logs, settings.

Access. Signed-in user whose email equals DIARI_ADMIN_EMAIL.

Main UI. Dashboard overview, User Management (list and read-only detail modal), Mindful Analytics, AI & External Services, Audit Logs, settings forms.

Actions. View users and aggregated journal metrics; test email; test AI; view logs; save settings (including optional token fields stored in system_settings). The admin UI does not provide disable-account or delete-account actions.

Server interaction. /api/admin/* endpoints.



8.13 PWA splash (pwa-splash.html)

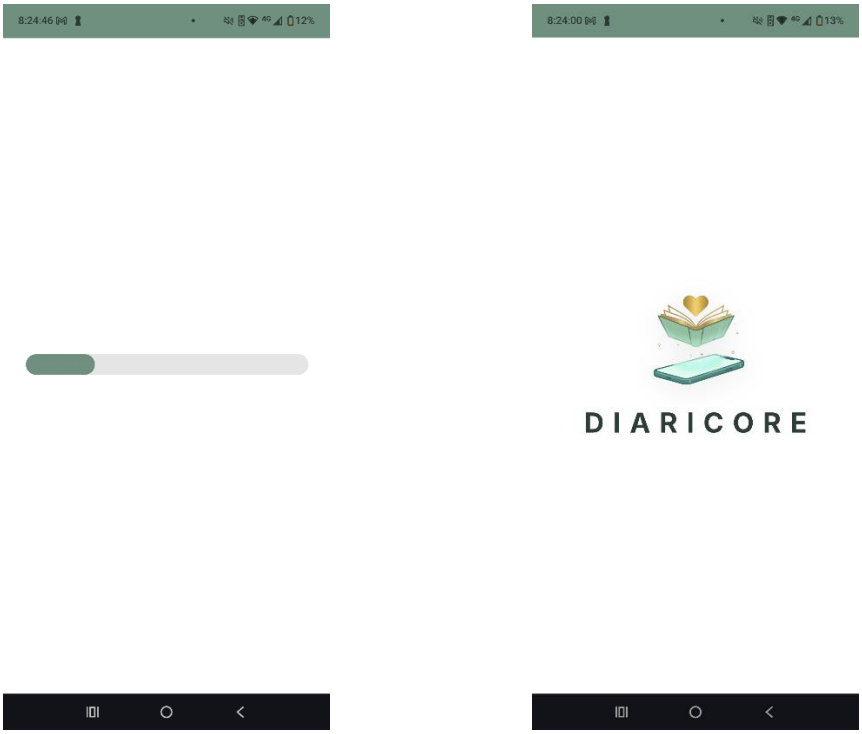
Purpose. Show a branded launch surface before the main shell paints.

Access. Installed PWA launch path as configured by client scripts.

Main UI. Logo/launch animation assets referenced by pwa-splash-boot.js and diari-pwa-launch.js.

Actions. Automatic continue into dashboard or login.

Server interaction. Static page plus service-worker navigation cache.



9. Authentication, Authorization, and Account Security

9.1 Sessions and authorization

After successful login (and TOTP if required), Flask stores user_id and a random csrf_token in the session. Cookie flags: HttpOnly, SameSite=Lax, 14-day permanent session. On Railway or FLASK_ENV=production, Secure is enabled. State-changing authenticated routes call _require_authenticated_user(), which rejects missing sessions and validates CSRF via the X-CSRF-Token header or a same-origin Origin check. Journal routes further require that entry.user_id matches the session. Admin routes additionally require the configured admin email. Disabled users cannot log in.

9.2 Passwords

Passwords are hashed with Werkzeug. New passwords must be 12–64 characters, include upper, lower, digit, and special character, contain no spaces, avoid a small common-password list, and must not contain the user's username, email, or names. The same rules run in password_policy.py and static/js/password-policy.js. Password reset and logged-in password change both require a Brevo OTP before the hash is updated.

9.3 Email OTP (Brevo) versus Google Authenticator

These are separate mechanisms. Brevo delivers six-digit codes for registration, password reset, email change, password change, and TOTP recovery. Codes expire (10 minutes for registration/reset; 15 minutes for TOTP recovery). Google

Authenticator (or any TOTP app) generates rotating codes from a shared secret stored on the user row. Authenticator codes are not sent by email during normal sign-in.

9.4 Google Authenticator TOTP

Purpose: require something the user has (the authenticator app) in addition to the password. From Profile, the user opens Authenticator setup, re-enters the password, and POST /api/user/totp/setup stores a pending totp_setup_secret. The API returns an otpauth URL, an SVG QR data URI (segno), and the secret for manual entry. The user scans the QR in Google Authenticator; the app generates 6-digit codes every 30 seconds. Confirming with POST /api/user/totp/confirm writes totp_secret and totp_enabled. Disable requires password plus a current TOTP code. If the phone is unavailable at login, recovery request emails a one-time code; verifying it signs the user in and turns TOTP off until they enroll again. Secrets are selected in SQL for auth but stripped from ordinary user JSON (except totpEnabled). The setup response necessarily includes the secret so the user can enroll; it must not be logged or pasted into documentation.

9.5 Other security controls

- Input normalization: journal text cannot contain < or >; nickname length 4–64; max 20 tags per entry.
- Headers: X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and a Content-Security-Policy unless DIARI_DISABLE_CSP is set.
- SECRET_KEY must be set in production; the code warns if the development default is still in use on Railway.

10. Rate Limiting

Rate limiting reduces brute-force and OTP-spam abuse. It is not applied to every route. Two layers exist: in-memory IP buckets (auth_security.rate_limit_check) and persistent per-account tables for login lockouts and OTP resends.

Scope	Limit	Window	User-visible effect
POST /api/register	8 requests	3600 s / IP	JSON error: Too many requests. Please wait a moment and try again.
Register verify & resend (IP)	10 requests	900 s / IP	Same generic too-many-requests message
POST /api/login and /api/login/totp (IP)	60 requests	900 s / IP	Generic too-many-requests message
Scope	**Limit**	**Window**	**User-visible effect**
Login failures (account key)	5 failures	900 s, then 900 s lock	Message with remaining attempts; then 15-minute lock
OTP resend per flow + account	5 sends	900 s, then 900 s lock	Too many OTP requests. Please try again later.
POST /api/password/forgot (IP)	30 requests	3600 s / IP	Generic too-many-requests message
POST /api/entries/analyze-text	40 requests	60 s / user	Generic too-many-requests message
POST /api/uploads/image	25 requests	60 s / user	Generic too-many-requests message
POST /api/voice/transcribe	15 requests	60 s / user	Generic too-many-requests message

In-memory IP limits reset if all Gunicorn workers restart. Login lockouts and OTP resend limits persist in PostgreSQL so they survive deploys. Journal GET/PATCH/DELETE list routes are not covered by these buckets.

11. Journal Data Management

Create: POST /api/entries stores title, text_content, tags_json, image_urls_json, entry_datetime_utc, and mood fields. Read: GET /api/entries (list) and GET /api/entries/<id>. Update: PATCH /api/entries/<id> (content and/or mood). Delete: DELETE /api/entries/<id>. Tags live in user_tags and as JSON on each entry; users can add icon names from the icon libraries. Streaks are computed on the client (diari-streak.js) from entry dates, not as a separate SQL table. Analytics on Insights and Dashboard are aggregations of stored emotion_label, scores, tags, and timestamps—there is no separate analytics warehouse.

12. Machine Learning: Dataset, Training, and Inference

12.1 Model architecture

The classifier is a Transformer encoder, not a classical model such as SVM or Naive Bayes. The backbone is XLM-RoBERTa-Base. The Hugging Face Space reconstructs a custom head matching the Colab notebook: Dropout(0.4) → Linear(768, 384) → LayerNorm → GELU → Dropout(0.2) → Linear(384, 5). Inference uses ONNX Runtime on CPU with max length 256 tokens. Labels in alphabetical index order are angry, anxious, happy, neutral, sad. Sentiment is derived: happy → positive; angry/anxious/sad → negative; otherwise neutral.

12.2 Dataset

The fine-tuning file provided with the project is 1500_dataset_expanded.xlsx (also uploaded in Colab as the training workbook). The notebook records shape (1593, 3). Columns used for modeling are text (journal-style content), label (emotion class), and language (english, filipino, taglish). word_count and sent_count are computed in the notebook, not stored as original columns. One language cell is recorded as “neutral” (a data quirk, n=1). Purpose: supply labeled bilingual/mixed journal-like sentences for five-class emotion classification.

label	Samples	Share of 1,593
neutral	334	21.0%
angry	330	20.7%
sad	330	20.7%
happy	300	18.8%
anxious	299	18.8%

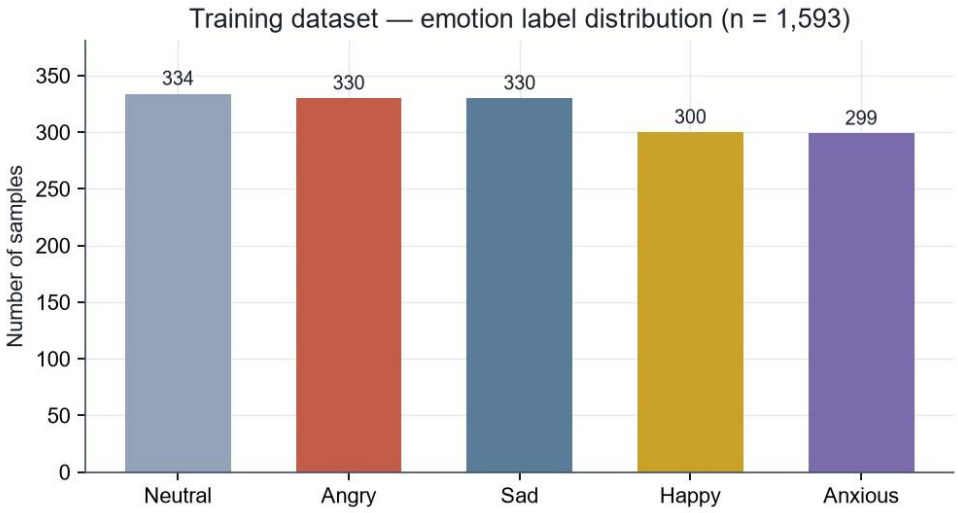


Figure 3. Emotion label counts in the training workbook (n = 1,593).

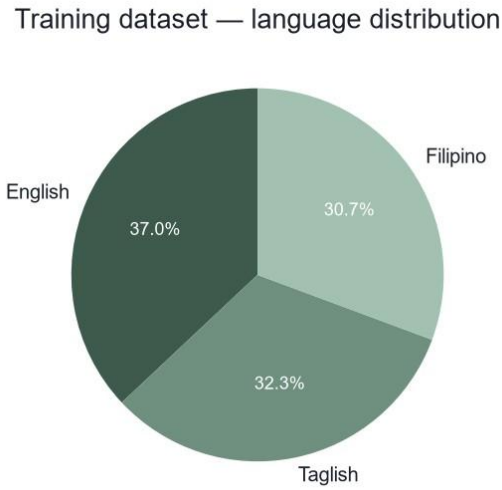


Figure 4. Language field distribution for english, taglish, and filipino. One workbook row with language = "neutral" (a data quirk, n = 1) is omitted from this chart so it is not shown as a language class.

Whole-row examples from the spreadsheet are omitted here. Many training sentences are long personal-style narratives; reproducing them would add little and risks copying sensitive-style content. The text field is the only content passed into tokenization.

Figure 5. Reference: Google Colab exploratory analysis of the uploaded workbook.

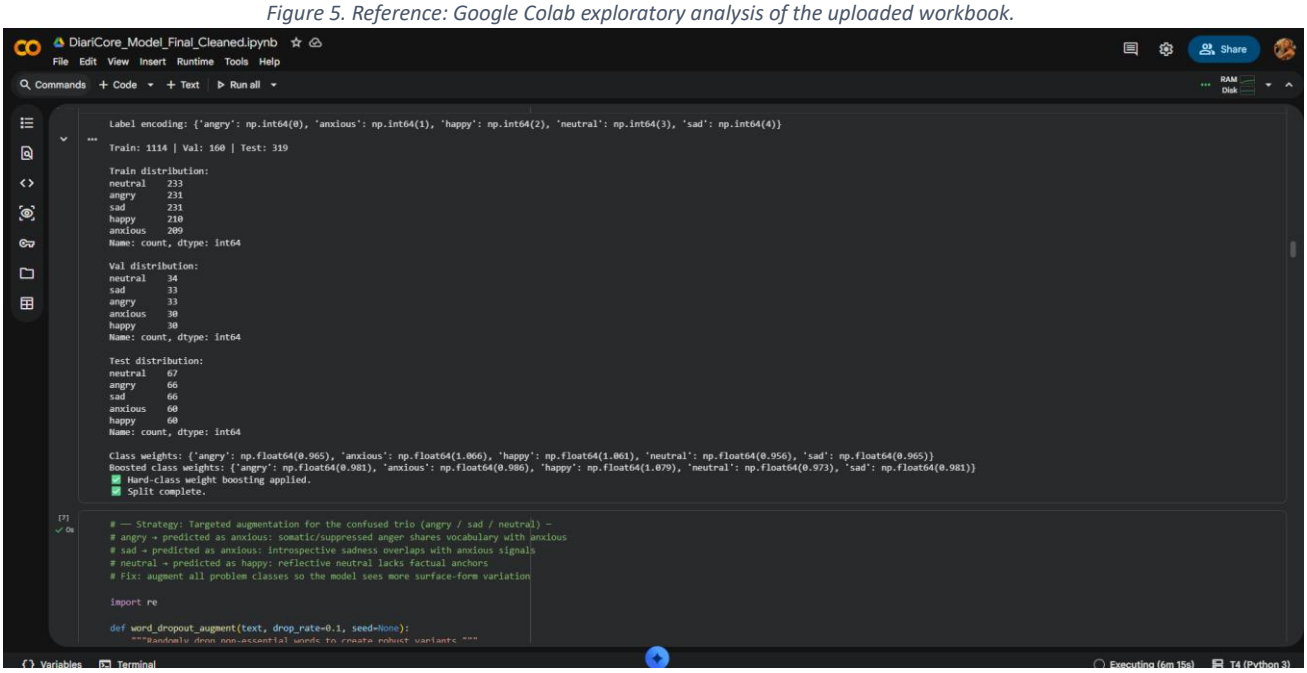


Figure 6. Reference: train/validation/test split cell in Colab.

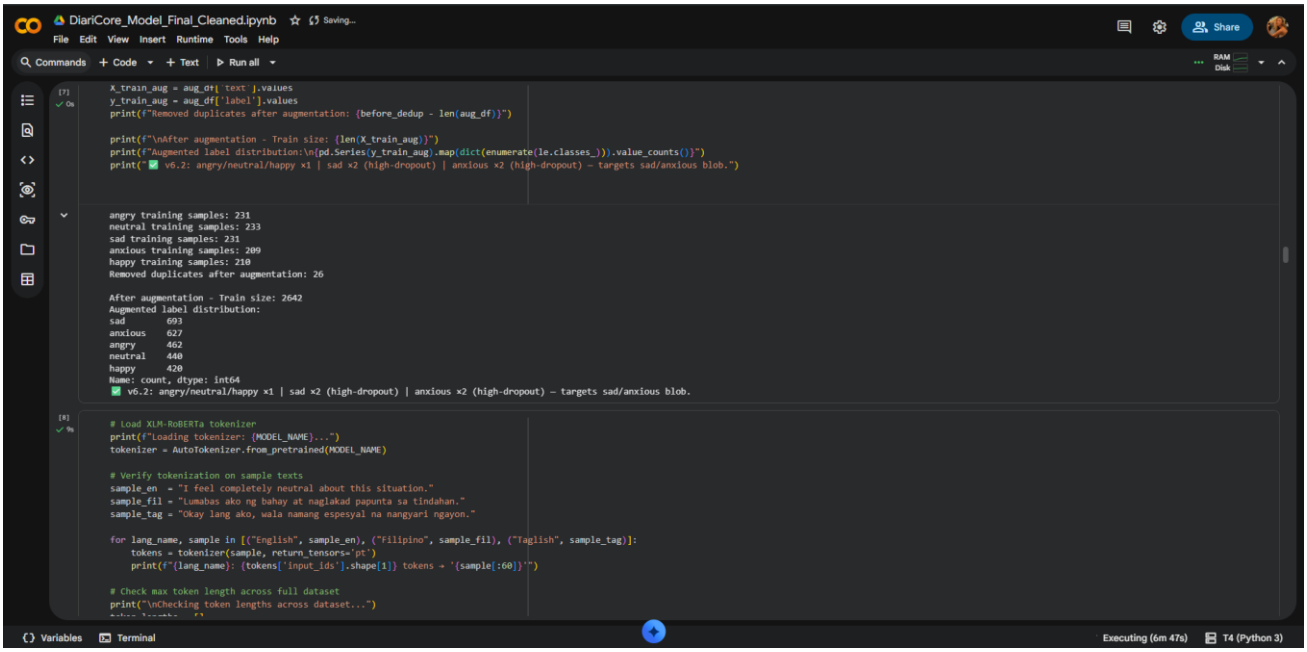


Figure 7. Reference: training-set augmentation cell in Colab.

12.3 Google Colab fine-tuning

Colab was used because a GPU runtime can fine-tune XLM-RoBERTa-Base without paying for a dedicated training cluster. The notebook tokenizes text with the XLM-RoBERTa tokenizer, trains for up to 6 epochs with early stopping, and saves Stochastic Weight Averaging (SWA) weights from later epochs. Training-set augmentation is applied in the notebook (duplicate-dropped). Artifacts are then uploaded to the Hugging Face Hub to replace oversized raw checkpoints.

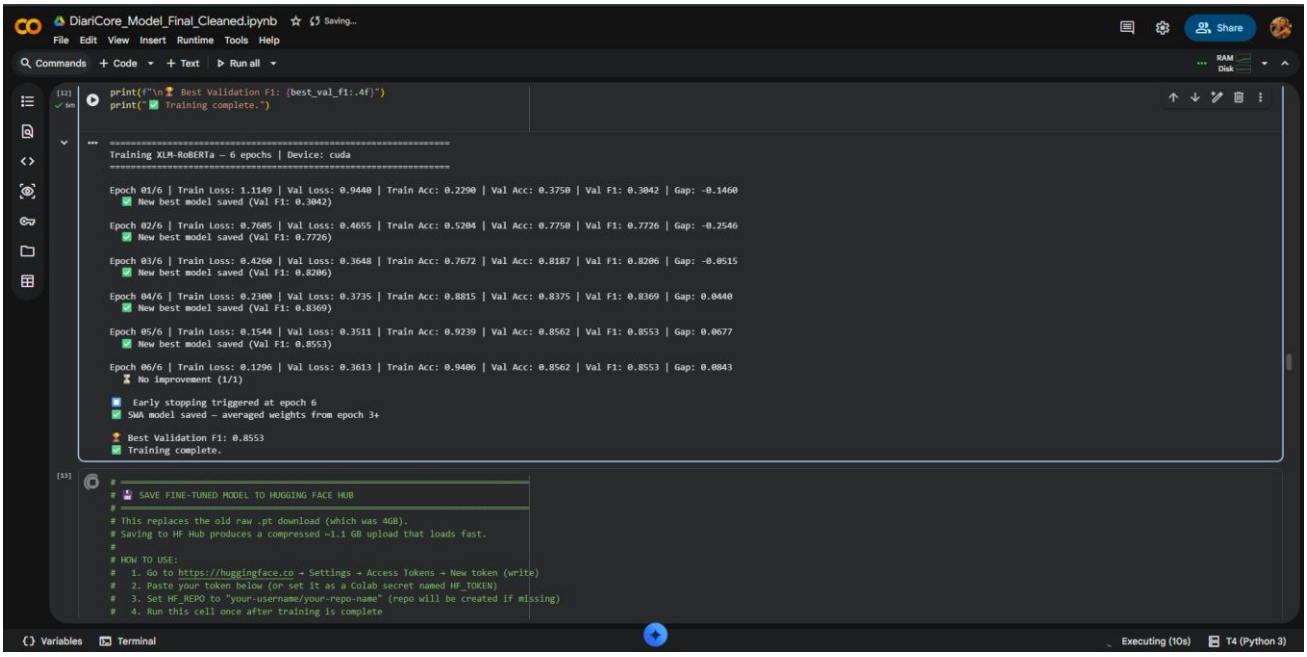


Figure 8. Reference Colab training log (one recorded run). That screenshot shows Best Validation F1 = 0.8356 and early stopping at epoch 5.

The notebook file stored in the repository contains a later evaluation cell. Those saved outputs report Best Validation F1 = 0.8413 (epoch 5 of a 6-epoch run that early-stopped at epoch 6), SWA weights from epoch 3+, and a held-out test summary: Test Accuracy 0.8370, Macro F1 0.8384, Weighted F1 0.8379, Macro Precision 0.8403, Macro Recall 0.8376, overfitting gap 0.0955, test support 319. Per-class test F1: angry 0.87, anxious 0.86, happy 0.85, neutral 0.87, sad 0.74. These numbers are notebook outputs, not live production accuracy on user journals.

Historical development configuration: a Colab screenshot of an earlier run lists Best Validation F1 0.8356.

Current notebook outputs in the repo list Best Validation F1 0.8413 and the test metrics above. Both describe XLM-RoBERTa fine-tuning on the same task; they should not be averaged or treated as a single official leaderboard score.

12.4 Export and Hub files

Confirmed files on Hugging Face Hub repository sseia/diari-core-mood: model.onnx (~1.11 GB), model_quantized.onnx (~279 MB), pytorch_model.bin (~1.11 GB), config.json, label_map.json, tokenizer.json, tokenizer_config.json, sentencepiece.bpe.model, special_tokens_map.json, README.md, .gitattributes. Production Space code prefers ONNX (HF_ONNX_FILE default model.onnx) and can skip PyTorch export when SKIP_ONNX_EXPORT is set.

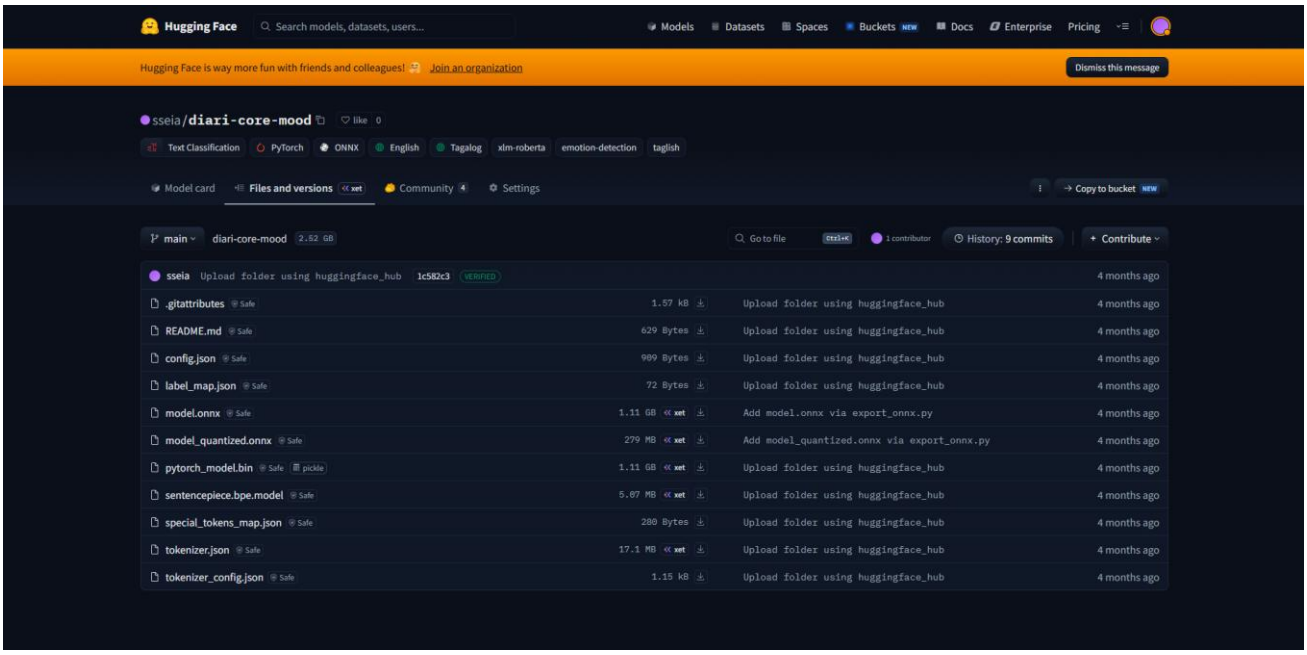


Figure 9. Historical Hub screenshot under sseia/diari-core-mood, showing the same artifact names (ONNX, quantized ONNX, PyTorch bin, tokenizer files).

Historical development configuration: model files were published at huggingface.co/sseia/diari-core-mood.

Current implementation: space_nlp.py and hf_space/app.py default to Hub id sseia/diari-core-mood and Space sseia/diaricore-inference (SPACE_URL default https://sseia-diaricore-inference.hf.space).

12.5 Hugging Face Space inference

hf_space/ is a Docker Space (Python 3.11, uvicorn on port 7860). On startup it warms the model on a background thread. Loading: download tokenizer snapshot (ignoring weight binaries), optionally download pytorch_model.bin and export ONNX, otherwise download model.onnx. Predict: tokenize → ONNX logits → softmax → per-class calibration multipliers → optional keyword override for low-confidence sad/anxious. Response keys include emotionLabel, emotionScore, sentimentLabel, sentimentScore, all_probs, engine (onnx-space or fallback), and timing.

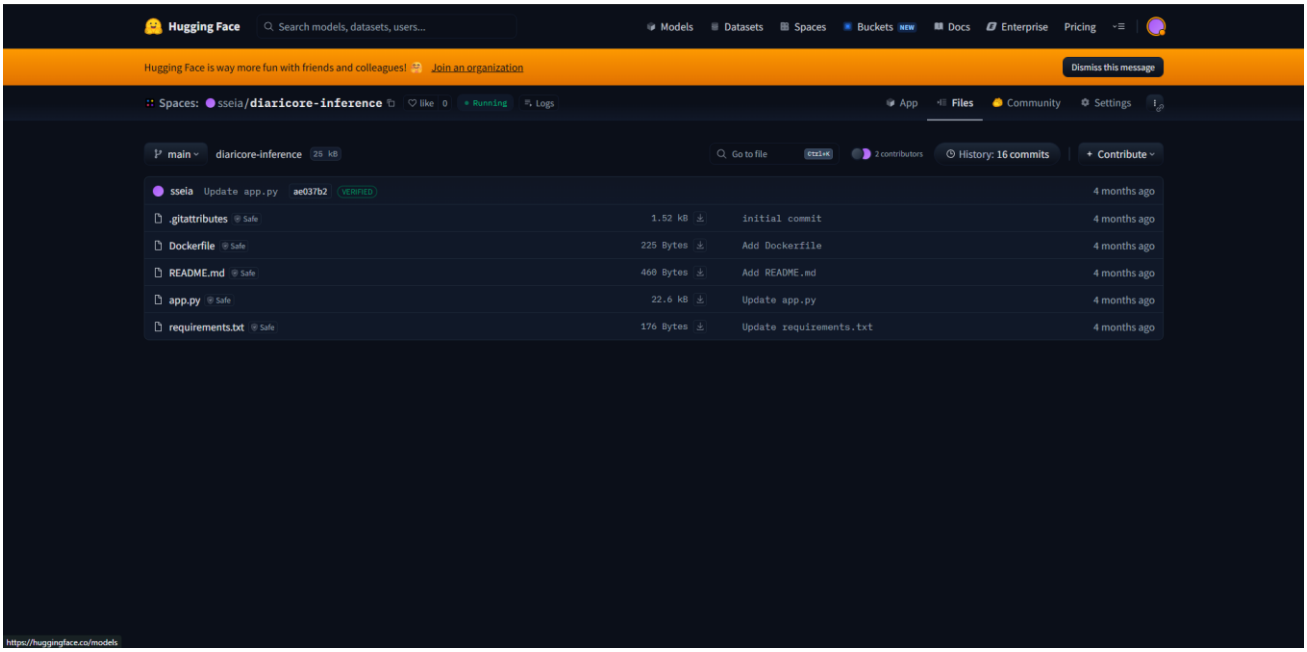


Figure 10. Historical Space screenshot for sseia/diaricore-inference (Docker app.py).

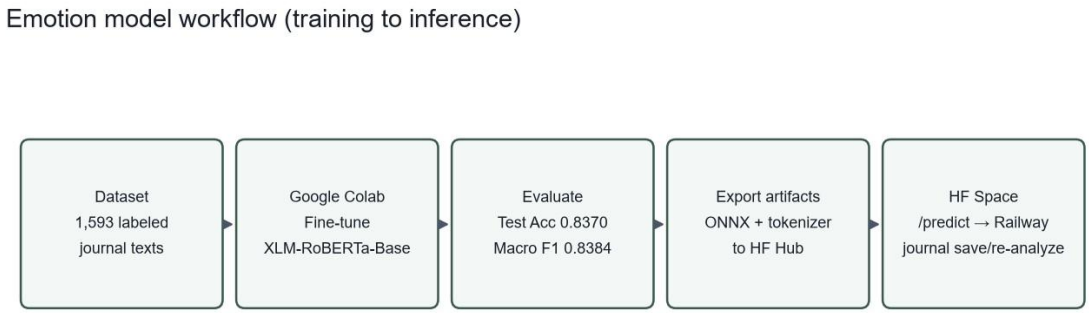


Figure 11. Training-to-inference workflow used by the current system.

13. Voice Functionality

Users open Voice Entry from Write Entry. The page asks for the microphone, renders a waveform, and—when SpeechRecognition or webkitSpeechRecognition exists—starts live captions after getUserMedia succeeds (this ordering avoids Chromium audio-capture failures). A MediaRecorder backup always records. English tends to use Xenova/whisper-tiny on-device; Filipino/Taglish uses Xenova/whisper-small with a Tagalog language hint. Transcript text is editable, word-counted, and copied into the journal textarea. Chrome and Edge are the practical browsers for Web Speech; other browsers may record audio but lack live captions.

POST /api/voice/transcribe remains available when a Hugging Face token is configured. It posts audio to the HF inference router using HF_SPEECH_MODEL (default openai/whisper-large-v3-turbo). This path is rate-limited and is not required for the primary on-device/Web Speech flow.

The voice functionality is operational and sufficiently reliable for normal use. However, it may still experience recognition inaccuracies under certain conditions. The feature remains an area for future refinement. Conditions that actually appear in the implementation and UX copy include: missing Web Speech support, microphone permission denial, short or silent recordings, first-load download of in-browser Whisper weights, and language mismatch between the selected voice locale and the user’s speech.

14. Progressive Web App, Caching, and Observed Performance

14.1 Manifest and installation

static/manifest.webmanifest is served at /manifest.webmanifest. It names the app DiariCore, uses display standalone, start_url /dashboard.html, portrait-primary, and 192/512/maskable icons. Shortcuts open Write Entry and Dashboard. pwa.js injects the manifest link, Apple web-app meta tags, registers /service-worker.js with scope /, and listens for beforeinstallprompt. On iOS Safari the UI instructs the user to use Share → Add to Home Screen. Installed mode is detected via display-mode: standalone or navigator.standalone.

14.2 Service worker and caching

The worker (cache name diaricore-pwa-v139 at the time of this writing) precaches the app shell: HTML pages, CSS/JS modules, logos, Lottie JSON, push templates, and the manifest. On fetch: API GET requests are never cached. Same-origin static assets and navigations use cache-first with network update; offline navigations fall back to cached dashboard or login. This is offline-tolerant for the shell and for drafts stored by diari-offline.js (localStorage/IndexedDB queues). It is not a complete offline clone of PostgreSQL. Mood analysis while offline uses a local estimate engine until sync.

14.3 Push in the worker

push events show notifications and POST /api/push/delivery-ack when the banner actually displays. notificationclick focuses an existing window or opens the target URL with pwa_fast=1. pushsubscriptionchange resubscribes.

14.4 Observed repeated-use performance

Observed behavior: the first few times the installed PWA opens, or the first time a given page/feature is used, the experience can feel slightly slower. Later visits to the same page feel noticeably faster. This is documented as an observation, not as a defect that prevents normal use.

Confirmed implementation that supports this observation: the service worker precache and cache-first strategy for HTML/CSS/JS; browser HTTP cache for CDN libraries (Bootstrap, fonts, transformers.js); transformers.js env.useBrowserCache = true for Whisper weights; PWA launch overlay only until diariPwaLaunchDone is set. API JSON is explicitly no-store, so faster repeat views are not explained by cached REST payloads. Hugging Face Space cold start (documented 30–60+ seconds on first /predict) can add delay to the first save/analyze after idle, which is separate from PWA asset caching. Railway free-tier process sleep can similarly delay the first HTTP request after inactivity. Those backend cold starts should not be confused with the client-side “second visit is faster” effect, though a user may feel both.

15. Push Notifications and Scheduled Jobs

DiariCore uses Web Push with VAPID (pywebpush), not a required Firebase project. The manifest includes Chrome’s well-known gcm_sender_id used by some browsers for Web Push compatibility; that value is not a private FCM server key.

Why a scheduler exists: browsers will not reliably fire “daily journal reminder” while the PWA is closed unless a push message arrives. The server therefore dispatches due notifications about once per minute. gunicorn.conf.py starts push_scheduler in each worker after fork (threads would not survive fork). The loop sleeps until the next minute boundary and calls dispatch_due_notifications. Daily reminders respect ui_preferences_json notification flags, reminderTimeOverride or a derived most-active hour, Asia/Manila date, and “already wrote today.” Streak-related evening nudges are described in PUSH_REMINDERS.md and template JSON.

cron-job.org is not required when the internal loop is running. It is the documented backup if DISABLE_INTERNAL_PUSH_CRON is true: a one-minute POST to /api/internal/push/dispatch with header X-Push-Cron-

Secret matching PUSH_CRON_SECRET. That is why an external cron service appears in the architecture: Railway's web dyno may be configured to disable in-process timers, or operators may want an external heartbeat if workers sleep.

16. External Services: Roles and Setup

16.1 Railway

Role: host the Flask web app, attach PostgreSQL, provide HTTPS at the *.up.railway.app domain, inject environment variables, and optionally mount a volume for /uploads. Setup: connect the GitHub repository, add a Postgres plugin (DATABASE_URL provided by Railway), set Variables, deploy using the Procfile. PORT is supplied by Railway; Gunicorn binds 0.0.0.0:$PORT. WEB_CONCURRENCY defaults to 1, matching free-tier CPU. The supplied Railway screenshot shows services Postgres and web online, volumes postgres-volume and web-volume, and hostname diari-core.up.railway.app on project daring-possibility.

Historical / screenshot hostname: diari-core.up.railway.app.

Current README and this document's primary live link: https://diaricore.up.railway.app/ . Treat the hyphenated hostname as a captured dashboard state; confirm the public URL in Railway's current domain settings if they differ.

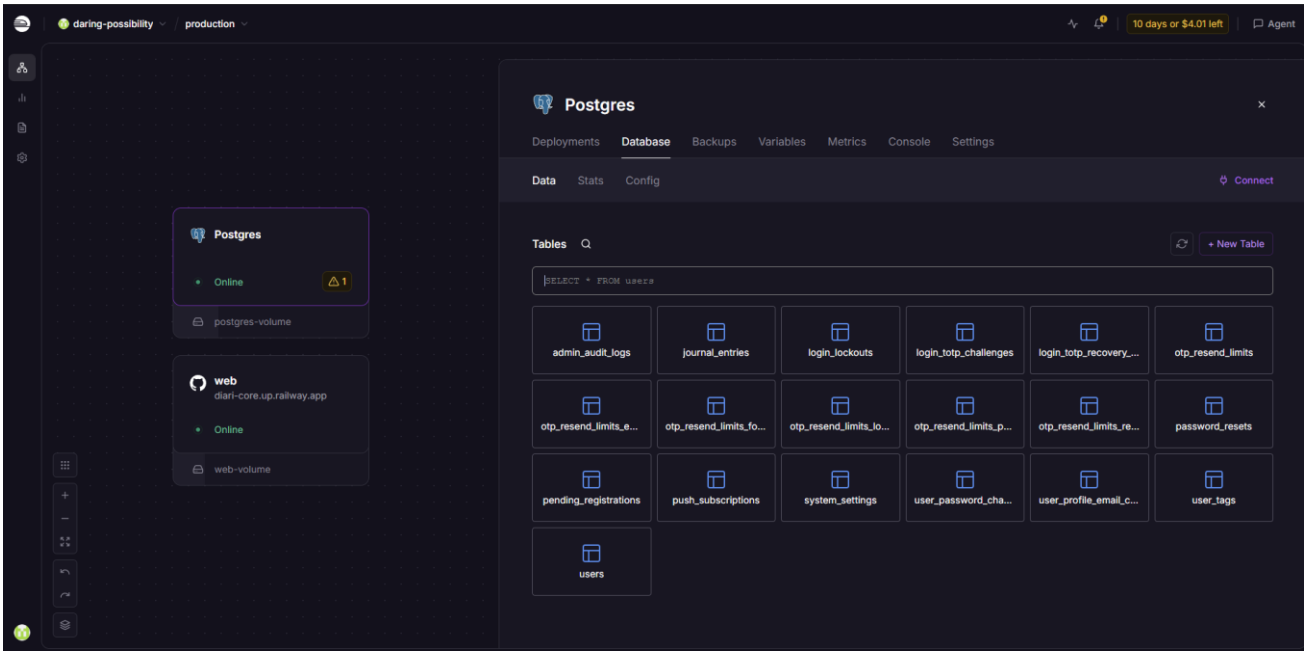


Figure 12. Railway production project with Postgres tables matching db.py (secrets in the UI are not reproduced).

16.2 Hugging Face

Role: store model artifacts (Hub) and run ONNX inference (Space). The Space does not need Railway RAM. Communication: Railway POST JSON to SPACE_URL/predict; no HF token is required on the web app for mood if the Space is public. Voice transcription is a different HF Inference API call and does need a token. Space env: HF_MODEL_ID, optional HF_TOKEN if the Hub repo is private, SKIP_ONNX_EXPORT, HF_ONNX_FILE. Repo hf_space/ is uploaded with scripts/upload_space.py.

16.3 Brevo

Role: transactional email for verification OTP, password reset, TOTP recovery, and the same templates reused for email/password change. Credentials come from the Brevo dashboard (API key and verified sender) and are stored as Railway variables or system_settings. The app POSTs to https://api.brevo.com/v3/smtp/email with header api-key. If enable_email_notifications is false, sends are skipped but flows still proceed. Locally, missing keys log the OTP instead of mailing it.

16.4 Google Authenticator

Not a hosted DiariCore service. Users install the app (or another TOTP client), scan the QR from Profile, and type codes at login. Validation is local to the Flask process via pyotp.

16.5 Cron-job service

Role: optional external minute tick for push dispatch. Configure URL https://<app>/api/internal/push/dispatch, method POST, header X-Push-Cron-Secret. Only needed if the internal scheduler is disabled or as redundancy.

16.6 Google Colab

Role: one-time / repeatable training environment. Not in the production request path. Notebook path: FinalProject_Resources/DiariCore_Model_Final_Cleaned.ipynb.

17. Environment Variables and Configuration

Values are read from the process environment (Railway Variables). Some keys can be overridden in system_settings for admin-operated deploys. Actual secrets are not listed.

Variable	Purpose	Used by	Value originates	How configured
DATABASE_URL	PostgreSQL connection	db.py	Railway Postgres plugin	Added automatically when the database service is linked; do not commit the URL
DATABASE_PATH	SQLite file locally	db.py	Developer choice	Set in the local shell / start-local.ps1
SECRET_KEY	Flask session signing	app.py	Application-generated random string	Paste into Railway Variables
PORT	HTTP listen port	gunicorn.conf.py / app.py	Railway runtime	Provided by the platform
WEB_CONCURRENCY	Gunicorn workers (default 1)	gunicorn.conf.py	Operator	Railway Variables if more workers are affordable
SPACE_URL	Mood Space base URL	space_nlp.py	Hugging Face Space	Set if not using the default sseia Space URL
BREVO_API_KEY	Brevo HTTP API key	app.py email helpers	Brevo dashboard	Railway Variables or admin system_settings
BREVO_SENDER_EMAIL	From address	app.py	Brevo verified sender	Railway Variables
BREVO_SENDER_NAME	From display name (default DiariCore)	app.py	Operator	Railway Variables
HF_API_TOKEN / HF_TOKEN	HF Inference for voice; Space/Hub if private	hf_speech.py, admin, upload_space.py	Hugging Face token UI	Railway Variables; never commit
HF_API_TOKEN_FOR_VOICE	Optional voice-only token	hf_speech.py	Hugging Face	Railway Variables
Variable	**Purpose**	**Used by**	**Value originates**	**How configured**

HF_SPEECH_MODEL	ASR model id	hf_speech.py	Operator (default Whisper turbo)	Optional override
VAPID_PUBLIC_KEY	Web Push public key	push_service.py, client subscribe	scripts/generate_vapid_keys.py	Railway Variables
VAPID_PRIVATE_KEY	Web Push signing key	push_service.py	Same generator run	Railway Variables (body or PEM)
VAPID_CLAIM_EMAIL	VAPID mailto claim	push_service.py	Operator email	Railway Variables
PUSH_CRON_SECRET	Authorize internal dispatch POST	app.py	Random string	Railway Variables; cron header
DISABLE_INTERNAL_PUSH_CRON	Skip in-process minute loop	push_scheduler.py	Operator	Set only if using external cron
UPLOADS_DIR	Persistent image directory	app.py	Railway volume mount path	Set to the mounted volume
DIARI_ADMIN_EMAIL	Admin UI allow-list	app.py	Operator	Railway Variables
ENTRY_WORD_MAX	Journal word cap (default 300; dataset-aligned; planned to raise later)	app.py	Operator	Optional
RAILWAY_ENVIRONMENT	Enables Secure cookies	app.py	Railway	Set by platform
DIARI_DISABLE_CSP	Disable CSP header	app.py	Operator	Only if a needed third-party script is blocked
HF_MODEL_ID	Hub repo for weights	hf_space/app.py	Hugging Face Hub	Space variables
SKIP_ONNX_EXPORT	Load Hub ONNX only	hf_space/app.py	Operator	Space variables for faster cold start
HF_ONNX_FILE	ONNX filename on Hub	hf_space/app.py	Hub file list	Space variables (model.onnx or model_quantized.onnx)

Historical Railway Variables screenshot: the production web service also listed HF_EMOTION_MODEL set to lproject012125/diari-core. That name is not referenced in the current Python codebase. Current Implementation: mood inference is pointed at SPACE_URL (default sseia Space) and the Space loads HF_MODEL_ID (default sseia/diari-core-mood). The screenshot still confirms Brevo, DATABASE_URL, HF tokens, VAPID keys, PUSH_CRON_SECRET, SECRET_KEY, UPLOADS_DIR, and DIARI_ADMIN_EMAIL as the live operator configuration pattern.

```
BREVO_API_KEY=YOUR_BREVO_API_KEY
DATABASE_URL=YOUR_DATABASE_CONNECTION_STRING
SPACE_URL=YOUR_HUGGING_FACE_ENDPOINT
SECRET_KEY=YOUR_FLASK_SECRET_KEY
VAPID_PRIVATE_KEY=YOUR_VAPID_PRIVATE_KEY
HF_API_TOKEN=YOUR_HUGGING_FACE_TOKEN
```

18. Local Development Setup

- Prerequisites: Python 3.10+ (3.12 recommended), Git, internet for Hugging Face Space mood calls.
- Clone the repository; python -m venv .venv; activate; pip install -r requirements.txt.

- Optional env: DATABASE_PATH=diaricore.local.db (SQLite). Leave DATABASE_URL unset. • Windows: powershell -ExecutionPolicy Bypass -File .\scripts\start-local.ps1
- Or: python app.py (listens on PORT or 5000).
- Verify GET http://127.0.0.1:5000/api/health then register and save an entry.
- Voice tests may need HF_API_TOKEN if using server transcription.
- Mood still uses the hosted Space; first analyze after Space sleep can take up to about a minute.

19. Railway Deployment

The GitHub repository is connected to a Railway project. A web service builds from the repo root, installs requirements.txt, and starts via Procfile: gunicorn app:app -c gunicorn.conf.py. Gunicorn timeout is 120 seconds to allow slow analyze calls. A Postgres plugin supplies DATABASE_URL. A volume should be mapped to UPLOADS_DIR so photos survive redeploys. Variables listed in chapter 17 are pasted from Brevo, Hugging Face, VAPID generation, and Railway's own database panel—never from git.

Free-tier considerations that shaped the architecture: limited RAM/CPU (hence HF inference), possible service sleep (first-request latency), usage credits (the Railway screenshot showed remaining trial/credit time), and a single Gunicorn worker by default. Hugging Face Spaces can pause (historical screenshot) or cold-start; the web app's keyword fallback keeps saves succeeding with reduced accuracy. Important operational practice: after changing VAPID keys, users must re-subscribe; after disabling internal cron, configure cron-job.org the same day or reminders stop.

20. Testing and Current Behavior

This table is a documentation review of the implemented system plus operator-described behavior. It is not a claim that every row was re-executed in an automated test suite during generation of this PDF. Where a feature is wired end-to-end in production code and described as used online, status is Operational. Known gaps use Operational — Known Limitations.

Test / feature	Expected result	Actual result (from implementation & ops notes)	Status
Registration	Pending user + email OTP	pending_registrations + Brevo or local dev log	Operational
Email verification	OTP creates users row	10-minute OTP; resend limits apply	Operational
Login	Session cookie after valid password	Lockout after 5 failures / 15 minutes	Operational
Google Authenticator 2FA	QR enroll; code required at login	pyotp window=1; recovery email disables 2FA	Operational
Rate limiting	Excess requests rejected with message	Selected routes only; IP buckets are in-memory	Operational — Known Limitations
Journal CRUD	Create/read/update/delete own entries	Ownership checks on GET/PATCH/DELETE	Operational
Database	Postgres in prod, SQLite local	DATABASE_URL switch in db.py	Operational
Emotion analysis	Labels stored on save	Space /predict; fallback on failure/cold start	Operational — Known Limitations
Voice	Transcript enters Write Entry	Reliable enough for normal use; not perfect	Operational — Known Limitations

PWA install	Standalone display, SW registered	Chrome/Edge/Android strong; iOS manual A2HS	Operational — Known Limitations
Offline drafts	Queue then sync	Local estimate mood until Space analysis	Operational — Known Limitations
Push notifications	Reminders on installed PWA	Needs VAPID, permission, dispatcher or cron	Operational — Known Limitations
Admin	Configured email reaches /admin; users can be viewed	DIARI_ADMIN_EMAIL allow-list; no disable/delete controls in the current UI	Operational
Suggestions page	Supportive activities and copy	Static template content; not a personalized recommender	Operational — Known Limitations
Test / feature	**Expected result**	**Actual result (from implementation & ops notes)**	**Status**
Live Railway app	HTTPS site serves login and APIs	Documented at diaricore.up.railway.app	Operational

21. Known Limitations and Current System Status

Area	Status
Core journaling + accounts	Fully functional in the current design
Emotion model	Functional; quality depends on Space availability and differs from training-set metrics on real mixed journals; sad class was the weakest in the notebook test report
Keyword fallback	Keeps the app up; less accurate than ONNX
Voice	Reliable enough for normal use; recognition can still be wrong; browser-dependent
PWA first load	Slightly slower until caches/weights populate; later use is faster
Offline	Shell + drafts; not full offline insights or server mood
Push	Installed PWA + OS permission; closed-app delivery depends on dispatcher remaining alive
Free-tier HF/Railway	Cold starts, possible Space pause, credit limits
Rate limits	Not universal; worker restart clears IP buckets
CSP	May need DIARI_DISABLE_CSP if a required CDN is blocked
ML as health advice	Explicitly out of scope — suggestions are not treatment
Journal word cap	Default 300 words, matching the training-set length limit; planned to expand with a longer dataset
Suggestions page	Functional, but content is currently static; personalization is a future improvement

22. Lessons Learned

- A 1 GB transformer does not belong on the same free web dyno as the user-facing API; splitting inference to Hugging Face was an availability decision, not a fashion choice.
- ONNX plus SKIP_ONNX_EXPORT reduces Space cold-start pain compared with exporting from PyTorch on every boot.
- Email OTP and TOTP solve different threats; recovery email that disables TOTP is a usability backdoor that must be rate-limited.
- Persistent lockout tables matter on platforms that restart processes often.
- Web Push without an always-on dispatcher is indistinguishable from “notifications are broken.”
- Service-worker cache-first makes repeat PWA use feel fast; it also means SW version bumps (diaricore-pwa-v*) are part of releasing frontend fixes.
- Starting Web Speech after getUserMedia is required on some Chromium builds.
- Environment variables must be treated as the real configuration; screenshots of dashboards go stale (Hub namespace, Railway hostname, paused Spaces).
- Training metrics from Colab are not production accuracy; fallback and calibration change live outputs.
- Privacy consent and XSS stripping on journal text are easier to enforce server-side than to retrofit.

23. Future Improvements

- Improve voice recognition robustness and first-load Whisper download UX.
- Reduce Space cold-start (quantized ONNX only, keep-alive, or paid Space).
- Richer offline support (queued analyze when back online is already partial; insights while offline are limited).
- Automated API and browser tests in CI.
- Accessibility pass (keyboard, contrast, screen readers) beyond current ARIA on selected controls.
- Raise or remove the 300-word journal cap after the training dataset covers longer entries.
- Replace the currently static Suggestions page with personalized, dynamically generated support and activity content.
- Expand training data for mixed Taglish journals and the weaker sad class.
- Optional stronger session policies (shorter TTL, device list).
- Documented runbooks for paused Spaces and exhausted Railway credits.

24. Project Links

Resource	URL
Live app (README)	https://diaricore.up.railway.app/
GitHub (this workspace origin)	https://github.com/lproject012125/diari-core
GitHub (README remote)	https://github.com/0323-3621-cell/diaricore
Model Hub (current code default)	https://huggingface.co/sseia/diari-core-mood
Inference Space (current code default)	https://huggingface.co/spaces/sseia/diaricore-inference

Portfolio and demo-video URLs were not present as dedicated fields in the repository; they are omitted rather than invented.

This document describes DiariCore as an integrated system: a Railway-hosted PWA and PostgreSQL application, a Colab-trained XLM-RoBERTa-Base emotion classifier served from Hugging Face, Brevo mail for OTP flows, device TOTP,

and a minute-level push dispatcher. Features that work with limitations are listed as such so the record stays accurate for repository readers, portfolio reviewers, and future maintainers.

25. Acknowledgements

DiariCore is documented here as a single-author project. The implementation, deployment, and this technical record are the work of Tolentino, Lawrence Dave P.

Special acknowledgment is extended to Jen Issa Mari B. Dimayacyac for her valuable and substantial contribution to the development of DiariCore, particularly in the machine-learning component of the system. She provided important assistance and guidance in the emotion-classification work, including aspects of the model-training workflow and related technical decisions, during a critical stage of the project's development. Her time, patience, technical insight, and willingness to provide support significantly contributed to strengthening the ML component of the system. The author is sincerely grateful for her involvement and recognizes her as an important person whose support and contribution played a meaningful role in the development of DiariCore.